

APRENDIZAJE DE MÁQUINAS (ACIF104) AVANCE INFORME FASE 2

Nombre integrante/s:

Javiera Chávez

Joaquín Olivares

Claudio Yáñez

**Curso: Aprendizaje
de Máquina**

Fecha: 28-07-2026

ÍNDICE

1.	Introducción	4
2.	Problemática y análisis de trabajos previos.....	5
2.1.	Problemática	5
2.2.	Análisis de trabajos previos	6
3.	Requisitos del proyecto.....	9
3.1.	Requisitos funcionales.....	9
3.2.	Requisitos no funcionales.....	10
4.	Técnicas de Aprendizaje Automático	11
4.1.	Selección de Arquitecturas de Deep Learning.....	13
4.2.	Comparación Sistemática de Técnicas Propuestas.....	14
4.3.	Arquitectura de sistema y Flujo de datos integral	16
5.	Metodología.....	17
6.	Dinámica de Trabajo y Planificación	18
5.1	Proyecto Sumativa 1: Fundamentos y Modelo Predictivo (17 de junio al 28 de julio del 2026)	18
7.	Desarrollo experimental.....	18
7.1.	Análisis Exploratorio de los datos.....	19
7.1.1.	Descripción del conjunto de datos.....	19
7.1.2.	Información general del conjunto de datos.....	19
7.1.3.	Estadísticas descriptivas	20
7.1.4.	Compleitud de los datos.....	20
7.1.5.	Distribución de las variables.....	21
7.1.6.	Identificación de valores atípicos.....	22
7.1.7.	Análisis de Correlación.....	23
7.2.	Calidad de los datos.....	24
7.2.1.	Compleitud.....	24
7.2.2.	Consistencia.....	24
7.2.3.	Exactitud.....	25
7.2.4.	Balance de clases.....	25
7.3.	Preprocesamiento de los datos.....	26
7.3.1.	Eliminación de variables no predictivas.....	26
7.3.2.	Normalización de nombres de variables.....	27
7.3.3.	Codificación de variables categóricas.....	27
7.3.4.	Separación de variables predictoras y variable objetivo.....	28
7.4.	Preparación para el entrenamiento.....	28
7.4.1.	División de los datos.....	28

7.4.2.	Escalado de las variables.....	28
7.4.3.	Balanceo de clases mediante SMOTE.....	29
7.5.	Entrenamiento de los modelos	29
7.6.	Evaluación de modelos.	30
7.6.1.	Reportes de clasificación.....	30
7.6.2.	Comparación de métricas de desempeño.....	31
7.6.3.	Matriz de confusión.....	31
7.6.4.	Curva ROC.....	32
7.6.5.	Selección del mejor modelo.....	33
7.7.	Interpretabilidad del modelo mediante SHAP.	33
7.7.1.	Inicialización de SHAP.....	33
7.7.2.	Importancia global de las variables.....	34
8.	Limitaciones del Modelo y Propuestas de Mejora	35
8.1.	Identificación de Limitaciones del Modelo Actual.....	35
8.2.	Propuestas de Mejora Fundamentadas.....	35
9.	Arquitectura Frontend/Backend	36
9.1.	Panel de control	36
9.2.	Carga de archivo CSV.....	37
9.3.	Predicción individual.....	38
9.4.	Explicabilidad SHAP.....	39
9.5.	Monitoreo	39
9.6.	Historial	40
9.7.	Propuesta de funcionamiento futuro	41
10.	Desarrollo Inicial del Proyecto	42
10.1.	Hoja de ruta.....	42
10.2.	Dinámica de Trabajo	43
11.	Conclusión	43
12.	Bibliografía.....	44

1. Introducción

El análisis de la información para tomar decisiones es extremadamente importante hoy en día, por lo que en este informe se tomará una problemática relacionada con una Base de Datos, también denominada BD la cual será tomada en cuenta en la realización de un análisis con diferentes métodos que tienen como objetivo entregar información relevante a los cuales se le aplicará una puntuación en su aprendizaje pues se espera que posteriormente sean utilizados en la creación de un MVP viable por medio de procesos de back y front-end. Los conceptos detallados a continuación son:

- Problemática y análisis previos.
- Requisitos del proyecto
- Requerimientos funcionales
- Requerimientos no funcionales

2. Problemática y análisis de trabajos previos.

2.1. Problemática

La transformación digital impulsada por la Industria ha incrementado de manera significativa la cantidad de información disponible para las organizaciones. En la actualidad, los procesos industriales incorporan sensores, dispositivos IoT y sistemas de monitoreo capaces de registrar continuamente variables como temperatura, presión, vibración, velocidad de rotación y desgaste de los equipos. Esta disponibilidad de datos ha permitido avanzar hacia modelos de gestión más inteligentes, donde las decisiones pueden sustentarse en información obtenida en tiempo real y en el análisis de datos históricos (Hamasha et al., 2025).

A pesar de estos avances tecnológicos, una gran parte de las organizaciones continúa utilizando estrategias tradicionales de mantenimiento, principalmente mantenimiento correctivo y mantenimiento preventivo. El mantenimiento correctivo consiste en intervenir un equipo únicamente después de que ocurre una falla, mientras que el mantenimiento preventivo programa inspecciones o reemplazos de componentes en intervalos previamente definidos. Si bien ambos enfoques ayudan a mantener la continuidad operacional, presentan limitaciones importantes, ya que pueden generar detenciones inesperadas, altos costos de reparación y reemplazos innecesarios de componentes que aún conservan vida útil (Benhanifia et al., 2025).

Desde el punto de vista económico, estas limitaciones tienen un impacto directo sobre la productividad de las organizaciones. Meitz et al. (2025) indican que las actividades de mantenimiento pueden representar entre un 15 % y un 70 % de los costos totales de producción, dependiendo del tipo de industria, mientras que las fallas inesperadas generan pérdidas asociadas a la interrupción de procesos, incumplimiento de plazos y utilización ineficiente de los recursos disponibles. En industrias altamente automatizadas, donde la continuidad operacional resulta crítica, incluso una detención de corta duración puede representar pérdidas económicas significativas.

Desde una perspectiva técnica, el principal desafío consiste en identificar oportunamente las condiciones que anteceden a una falla. Las variables operacionales registradas por sensores contienen información valiosa sobre el comportamiento de los equipos; sin embargo, debido al gran volumen de datos generado diariamente, resulta prácticamente imposible que dicho análisis sea realizado de forma manual. En consecuencia, las organizaciones requieren herramientas capaces de procesar automáticamente esta información y detectar patrones que permitan anticipar posibles fallas antes de que ocurran (Guidotti et al., 2025).

En respuesta a esta problemática surge el mantenimiento predictivo (*Predictive Maintenance*), estrategia que utiliza los datos históricos y variables de operación para estimar la probabilidad de falla de un equipo y planificar las intervenciones de mantenimiento en el momento más oportuno. Diversas investigaciones coinciden en que este enfoque permite reducir los tiempos de inactividad, optimizar la utilización de recursos y mejorar la disponibilidad de los activos industriales, constituyéndose actualmente en una de las principales aplicaciones del aprendizaje automático dentro de la Industria 4.0 (Benhanifia et al., 2025; Hamasha et al., 2025).

Sin embargo, la literatura también evidencia que la implementación de soluciones de mantenimiento predictivo enfrenta desafíos importantes. Guidotti et al. (2025) destacan la escasez de conjuntos de datos industriales públicos, la dificultad para generalizar modelos entre distintos entornos productivos y la necesidad de desarrollar soluciones más interpretables para los usuarios finales. Asimismo, Hamasha et al. (2025) señalan que la efectividad de estos sistemas depende no solo del desempeño del modelo predictivo, sino también de su integración con plataformas de monitoreo que permitan apoyar la toma de decisiones de manera oportuna.

Considerando estos antecedentes, el presente proyecto propone el desarrollo de una plataforma de mantenimiento predictivo denominada Perfectime, cuyo propósito es integrar técnicas de aprendizaje automático supervisado con una interfaz orientada a supervisores y responsables de mantenimiento. La solución buscará analizar datos provenientes de equipos industriales, estimar la probabilidad de falla y entregar información clara que facilite la planificación de las actividades de mantenimiento, contribuyendo a una gestión más eficiente de los activos industriales.

2.2. Análisis de trabajos previos.

Se realizó una revisión de cinco investigaciones científicas recientes publicadas durante el año 2025. La selección de estos trabajos permitió analizar las principales tendencias del área, las tecnologías utilizadas y los desafíos que aún enfrenta la implementación de soluciones de mantenimiento predictivo en entornos industriales.

En términos generales, los estudios revisados coinciden en que el mantenimiento predictivo representa una evolución respecto de las estrategias tradicionales de mantenimiento correctivo y preventivo. Mientras los enfoques convencionales reaccionan ante una falla o establecen intervenciones periódicas basadas en calendarios, el mantenimiento predictivo utiliza información proveniente de sensores y registros históricos para anticipar el comportamiento futuro de los equipos y planificar oportunamente las intervenciones necesarias (Benhanifia et al., 2025; Meitz et al., 2025).

Uno de los trabajos corresponde a la revisión sistemática desarrollada por Guidotti et al. (2025), quienes analizaron 216 investigaciones relacionadas con la aplicación de técnicas supervisadas de aprendizaje automático en mantenimiento predictivo. Los autores concluyen que algoritmos como Random Forest, Support Vector Machine, Gradient Boosting y redes neuronales son actualmente los más utilizados debido a su capacidad para identificar patrones complejos de falla a partir de datos históricos. Sin embargo, también señalan que gran parte de las investigaciones presentan limitaciones asociadas a la disponibilidad de conjuntos de datos públicos, la dificultad para generalizar los modelos entre distintos escenarios industriales y la escasa incorporación de mecanismos que permitan explicar las decisiones tomadas por los algoritmos. Estos aspectos resultan especialmente relevantes para este proyecto, ya que evidencian la necesidad de desarrollar soluciones que no solo alcancen un buen desempeño predictivo, sino que además entreguen resultados comprensibles para los usuarios finales.

Por otro lado, Meitz et al. (2025), realizaron una revisión de 249 publicaciones científicas con el objetivo de identificar tendencias y desafíos abiertos en el mantenimiento predictivo. Los autores destacan que las organizaciones han incrementado el interés por estas tecnologías debido al impacto económico que generan las fallas inesperadas y a la necesidad de mejorar la disponibilidad de los equipos industriales. No obstante, advierten que aún existen dificultades relacionadas con la integración de múltiples fuentes de datos, la ausencia de metodologías estandarizadas para comparar soluciones y la complejidad de implementar estos sistemas en ambientes industriales reales.

Por su parte, Benhanifia et al. (2025) comparan las distintas estrategias de mantenimiento utilizadas actualmente en la industria manufacturera mediante una revisión sistemática basada en la metodología PRISMA. Los autores concluyen que el mantenimiento predictivo ofrece ventajas significativas respecto de los enfoques correctivo y preventivo, principalmente debido a la disminución de tiempos de inactividad, la optimización de recursos y la reducción de costos operacionales. Además, destacan que empresas como IBM, Siemens y ABB ya utilizan soluciones de este tipo, lo que demuestra que el mantenimiento predictivo constituye una tecnología madura y con aplicaciones reales dentro de la industria.

A diferencia de los estudios anteriores, que se centran en revisiones bibliográficas, Aminzadeh et al. (2025) presentan un caso práctico donde desarrollan un sistema de mantenimiento predictivo para compresores industriales integrando sensores, bases de datos y modelos de aprendizaje automático. Este trabajo demuestra que el valor de estas soluciones no depende únicamente del algoritmo utilizado, sino también de la capacidad para integrar la adquisición de datos, el procesamiento de información y la generación de alertas dentro de una plataforma funcional que apoye la operación diaria de los usuarios.

Finalmente, Hamasha et al. (2025) proponen un marco integral para el desarrollo de sistemas de mantenimiento predictivo basados en Internet de las Cosas, Edge Computing e Inteligencia Artificial. Los autores plantean que el éxito de este tipo de soluciones depende de la interacción entre sensores, infraestructura tecnológica, procesamiento de datos y herramientas que faciliten la toma de decisiones. Asimismo, identifican desafíos relacionados con la interoperabilidad entre plataformas, la seguridad de la información y la calidad de los datos utilizados para entrenar los modelos predictivos.

A partir del análisis realizado es posible identificar varios elementos comunes entre las investigaciones revisadas. En primer lugar, existe consenso en que el mantenimiento predictivo constituye una estrategia efectiva para disminuir costos y aumentar la disponibilidad de los equipos industriales. En segundo lugar, la mayoría de los estudios utiliza técnicas de aprendizaje automático supervisado para desarrollar modelos de predicción de fallas. Finalmente, todos los trabajos coinciden en que la implementación de estas soluciones aún enfrenta desafíos asociados a la disponibilidad de datos, la interpretabilidad de los modelos y su integración dentro de sistemas de software que apoyen la toma de decisiones en entornos reales.

En este contexto, el presente proyecto busca aportar mediante el desarrollo de Perfectime, una plataforma orientada a supervisores y responsables de mantenimiento que integre un modelo de aprendizaje automático supervisado con herramientas de visualización y monitoreo. A diferencia de los trabajos centrados exclusivamente en evaluar el desempeño de los algoritmos, la propuesta considera el desarrollo de una solución informática que facilite la interpretación de las predicciones y apoye la planificación de las actividades de mantenimiento, respondiendo a varios de los desafíos identificados en la literatura reciente.

3. Requisitos del proyecto

A partir de la problemática identificada y del análisis de los trabajos previos, se definieron los requisitos funcionales y no funcionales para Perfectime, una plataforma orientada al análisis predictivo de fallas en equipos industriales mediante técnicas de aprendizaje automático. Los requisitos propuestos consideran las características del conjunto de datos AI4I 2020 Predictive Maintenance Dataset, el cual contiene registros históricos de variables operacionales asociadas al funcionamiento de equipos industriales y sus respectivos eventos de falla.

3.1.Requisitos funcionales.

Los requisitos funcionales describen las principales funcionalidades que deberá ofrecer la plataforma para cumplir con los objetivos del proyecto.

ID	Requisito funcional	Justificación
RF-01	El sistema deberá permitir importar archivos en formato CSV que contengan registros históricos de funcionamiento de equipos industriales.	El dataset AI4I 2020 se distribuye en este formato y constituye la fuente de datos utilizada por el sistema.
RF-02	El sistema deberá validar la estructura e integridad de los datos antes de realizar el análisis.	Evita errores ocasionados por registros incompletos o inconsistentes durante el procesamiento.
RF-03	El sistema deberá procesar las variables operacionales del conjunto de datos (temperatura del aire, temperatura del proceso, velocidad de rotación, torque y desgaste de la herramienta) para generar predicciones.	Estas variables representan los atributos utilizados por el modelo de aprendizaje automático.
RF-04	El sistema deberá predecir la probabilidad de que un equipo presente una falla utilizando un modelo de aprendizaje automático previamente entrenado.	Corresponde a la funcionalidad principal de la plataforma.
RF-05	El sistema deberá indicar el tipo de falla asociado cuando exista una condición de falla TWF(<i>Tool Wear</i>), HDF(<i>Heat Dissipation</i>), PWF(<i>Power Failure</i>), OSF(<i>Overstrain</i>) o RNF(<i>Random Failure</i>).	El dataset incorpora diferentes categorías de fallas que pueden ser utilizadas para apoyar el diagnóstico.
RF-06	El sistema deberá mostrar los resultados del análisis mediante una interfaz gráfica que facilite su interpretación por parte del usuario.	Permite apoyar la toma de decisiones durante la planificación del mantenimiento.

RF-07	El sistema deberá permitir consultar los registros analizados y las predicciones generadas.	Facilita el seguimiento y comparación de resultados obtenidos durante el análisis.
RF-08	El sistema deberá generar reportes con las predicciones obtenidas para apoyar la planificación de actividades de mantenimiento.	Permite documentar los resultados y facilitar la toma de decisiones.
RF-09	El sistema deberá permitir administrar usuarios mediante autenticación básica y distintos niveles de acceso.	Resguarda la información del sistema y restringe el acceso a funciones administrativas.

Tabla 1. Requisitos funcionales.

3.2.Requisitos no funcionales

Los requisitos no funcionales establecen las características de calidad que deberá cumplir la solución propuesta para garantizar una adecuada experiencia de uso y facilitar su futura evolución.

Categoría	Requisito	Justificación
Usabilidad	La plataforma deberá presentar una interfaz intuitiva que permita interpretar fácilmente las predicciones y resultados obtenidos.	Los trabajos revisados destacan la importancia de que los sistemas sean comprensibles para el usuario final (Hamasha et al., 2025).
Explicabilidad	El sistema deberá mostrar las variables que más influyen en la predicción realizada por el modelo cuando sea técnicamente posible.	La interpretabilidad constituye uno de los desafíos identificados por Guidotti et al. (2025).
Escalabilidad	La arquitectura deberá permitir incorporar nuevos conjuntos de datos y actualizar el modelo predictivo sin modificar la estructura principal del sistema.	Facilita la evolución futura del software y su adaptación a distintos escenarios industriales.
Seguridad	El acceso a la plataforma deberá realizarse mediante autenticación de usuarios y control de permisos.	Protege la información procesada por el sistema.
Confiabilidad	El sistema deberá entregar resultados consistentes a partir de datos previamente validados y procesados.	Reduce el riesgo de generar predicciones basadas en información incorrecta.

Monitoreabilidad	La plataforma deberá registrar las predicciones realizadas, los archivos procesados y los eventos relevantes del sistema.	Facilita el seguimiento del funcionamiento y la detección de posibles errores.
Rendimiento	El tiempo requerido para procesar un conjunto de datos y generar las predicciones deberá ser adecuado para no afectar la experiencia del usuario.	Favorece la utilización práctica de la plataforma.
Mantenibilidad	El software deberá desarrollarse utilizando una arquitectura modular que facilite futuras mejoras o incorporación de nuevos algoritmos.	Permite mantener y evolucionar el sistema con menor esfuerzo.
Portabilidad	La aplicación deberá ejecutarse correctamente en diferentes equipos que cuenten con el entorno de desarrollo definido para el proyecto.	Facilita su implementación en distintos computadores durante el desarrollo y las pruebas.

Tabla 2. Requisitos No Funcionales.

4. Técnicas de Aprendizaje Automático

Para el desarrollo del núcleo predictivo de *Perfectime*, enfocado en la detección anticipada de fallas del *dataset* industrial AI4I 2020, se evaluaron e integraron tres algoritmos de Aprendizaje Automático Tradicional ampliamente respaldados por la literatura científica.

Técnica	Tipo	Justificación de elección
Random Forest	Ensamblado (Árboles)	Este algoritmo de ensamble basado en múltiples árboles de decisión combina bagging y selección aleatoria de características. Es adecuado para este proyecto debido a su capacidad para manejar relaciones no lineales complejas entre los datos de los sensores de telemetría, su resistencia al sobreajuste y su habilidad para proporcionar una medida directa de la importancia de las variables, lo que facilitará la interpretación técnica de las fallas(Breiman, 2001).

Support Vector Machines (SVM)	Hiperplano (Kernel).	Mediante el uso de funciones Kernel (como RBF), SVM permite proyectar los datos a espacios de mayor dimensión para encontrar hiperplanos de separación óptimos entre clases. Se incluye en este estudio por su solidez teórica y buen desempeño en problemas de clasificación binaria cuando las fronteras de decisión son no lineales, sirviendo como un punto de comparación robusto frente a los métodos basados en árboles (<i>Cortes & Vapnik, 1995</i>).
XGBoost / LightGBM	Boosting (Gradiente)	Como implementación optimizada de Gradient Boosting, XGBoost destaca por su alto rendimiento, velocidad de ejecución y manejo eficiente del desbalanceo de clases mediante ponderación de instancias y regularización. Su selección se justifica por ser el estado del arte en datos tabulares industriales, permitiendo capturar patrones finos en las variables físicas antes de que ocurra una avería. (Chen & Guestrin, 2016)

4.1. Selección de Arquitecturas de Deep Learning

Aunque las técnicas tradicionales de Machine Learning son eficaces para datos tabulares, las arquitecturas de Deep Learning ofrecen capacidades avanzadas para capturar patrones no lineales complejos y dependencias temporales en datos de monitoreo industrial:

- Perceptrón Multicapa (MLP / Feedforward Neural Network):

Justificación: Permite modelar interacciones complejas e interdependencias no lineales entre variables continuas (temperatura, torque, velocidad) sin necesidad de ingeniería de atributos manual extensa (Goodfellow et al., 2016).

- Redes Neuronales Convolucionales 1D (1D-CNN):

Justificación: Adecuadas para la extracción de características locales en series temporales o registros secuenciales de sensores. Capturan fluctuaciones abruptas en variables como torque y desgaste de herramienta antes de que ocurra una falla.

- Long Short-Term Memory (LSTM):

Justificación: Diseñadas para manejar dependencias de largo plazo en datos secuenciales. En un entorno operacional real, las fallas mecánicas son el resultado de la degradación progresiva en el tiempo; las redes LSTM permiten modelar este proceso de degradación acumulativa

4.2.Comparación Sistemática de Técnicas Propuestas

Técnica / Algoritmo	Ventajas	Desventajas	Costo Computacional	Interpretabilidad	Precisión Esperada (PR-AUC)	Aplicabilidad al Dataset AI4I 2020
Random Forest (ML Tradicional)	Robusto frente al ruido y sobreajuste; no requiere escalado estricto; maneja interacciones no lineales entre sensores.	Menor capacidad predictiva en patrones complejos comparado con Boosting; consume memoria en árboles muy profundos.	Bajo - Medio (Inferencia muy rápida)	Alta (Feature Importance directo)	Media - Alta (~0.88 - 0.92)	Baseline robusto para evaluar las relaciones iniciales entre torque, temperatura y velocidad.
SVM (Kernel RBF) (ML Tradicional)	Efectivo en alta dimensión; robusto si los límites de separación entre fallas son claros.	Muy sensible al escalado de características y al desbalanceo de clases; lento en datasets grandes.	Medio - Alto (Escala cuadráticamente)	Baja (Caja negra por transformación RBF)	Media (~0.82 - 0.86)	Requiere estandarización estricta (StandardScaler) en variables de proceso con rangos diversos.
XGBoost (Ensemble Boosting)	Estado del arte en datos tabulares; incluye regularización L1/L2 y manejo de desbalance con scale_pos_weight.	Requiere ajuste fino de hiperparámetros; propenso a sobreajuste si no se limita la profundidad.	Medio (Optimizado para CPU/GPU)	Media - Alta (Explicable vía SHAP)	Muy Alta (~0.94 - 0.97)	Técnica principal: Captura eficazmente las firmas de fallas complejas y desbalanceadas (TWF, HDF, PWF, OSF, RNF).

MLP (<i>Deep Learning</i>)	Aprende representaciones no lineales complejas directamente de los datos de sensores.	Requiere gran volumen de datos para no sobre ajustar; exige ajuste de capas y neuronas.	Medio - Alto (<i>Múltiples épocas de entrenamiento</i>)	Baja (<i>Caja negra, requiere LIME/SHAP</i>)	Alta (~0.90 - 0.93)	Evaluada para capturar dependencias cruzadas de múltiples sensores sin ingeniería previa de atributos.
1D-CNN (<i>Deep Learning</i>)	Extrae patrones y características locales automáticamente mediante filtros de convolución 1D.	Diseñada para secuencias; propensa a memorizar patrones si los datos tabulares son estáticos.	Alto (<i>Operaciones convolucionales más pesadas</i>)	Baja (<i>Filtros difíciles de interpretar</i>)	Alta (~0.99 - 0.92)	Aplicable si las lecturas de los sensores se agrupan en ventanas secuenciales de operación.
LSTM (<i>Deep Learning</i>)	Captura dependencias temporales a largo plazo y la acumulación paulatina de desgaste o estrés.	Costosa computacionalmente; entrenamiento lento; requiere secuencias temporales ordenadas.	Muy Alto (<i>Procesamiento recurrente</i>)	Baja (<i>Celdas de memoria no interpretables</i>)	Muy Alta (~0.92 - 0.95)	Idónea para modelar de forma acumulativa la variable de desgaste de herramienta (<i>Tool Wear [min]</i>).

4.3.Arquitectura de sistema y Flujo de datos integral



La arquitectura del sistema *Perfectime* ha sido diseñada bajo una estructura modular de seis etapas consecutivas, garantizando un flujo end-to-end estandarizado desde la captura de telemetría hasta la persistencia histórica de los eventos:

Entrada de Datos: Corresponde al punto de ingesta inicial donde se reciben las mediciones de los sensores industriales (temperatura, velocidad de rotación, torque y desgaste de herramientas) provenientes del *dataset* o flujos de transmisión IoT.

Preprocesamiento: Implementa un *pipeline* automatizado de tratamiento de datos responsable de validar la integridad del registro, aplicar normalización mediante *StandardScaler* y solucionar el desbalanceo severo de clases mediante la técnica *SMOTE*.

Modelo IA (Núcleo Predictivo): Modulo encargado de la inferencia utilizando el algoritmo seleccionado (XGBoost). Este bloque evalúa los patrones no lineales para determinar la probabilidad de falla (0/1), categorizar el tipo de avería específica (TWF, HDF, PWF, OSF, RNF) y calcular la explicabilidad local del resultado mediante valores *SHAP*.

Predicción (Capa API REST): Funciona como middleware orquestador que transforma los resultados del modelo en cargas útiles estandarizadas (respuestas JSON), gestiona los protocolos de autenticación y comunica de forma segura la lógica de negocio con las capas externas.

Interfaz de Usuario (Front-End): Panel de control interactivo diseñado para supervisores de planta, donde se visualizan alertas de riesgo operacional en tiempo real, métricas de rendimiento y gráficos explicativos de los factores que causan la predicción.

Almacenamiento (Base de Datos): Capa de persistencia en una base de datos relacional (SQL) que guarda el registro histórico de telemetría, las predicciones emitidas, logs de auditoría e indicadores de disponibilidad para análisis posteriores.

5. Metodología.

Para el desarrollo de este proyecto se utilizó la metodología CRISP-DM (Cross Industry Standard Process for Data Mining), ya que entrega una forma ordenada de abordar proyectos de minería de datos y aprendizaje automático. Esta metodología divide el trabajo en seis etapas: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue.

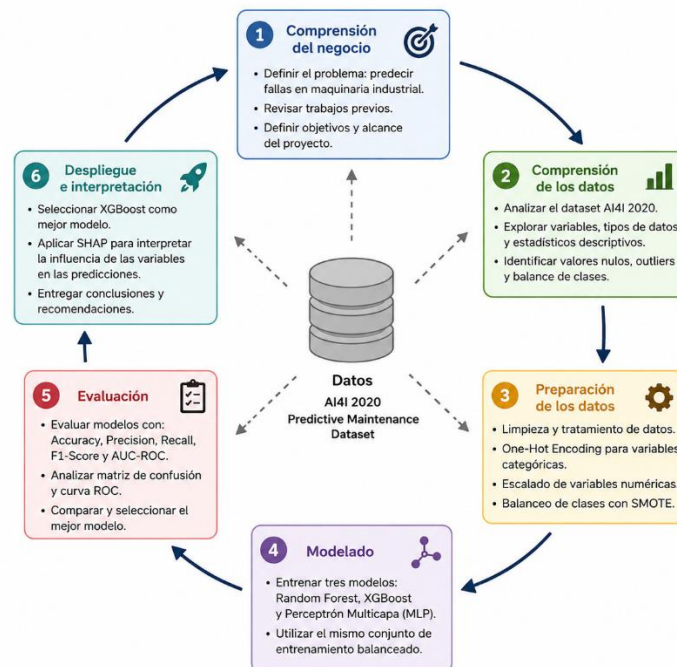
En primer lugar, se definió el problema que se buscaba resolver, el cual consistió en predecir fallas en maquinaria industrial utilizando el dataset AI4I 2020 Predictive Maintenance Dataset. Además, se revisaron antecedentes relacionados con el mantenimiento predictivo y se establecieron los objetivos que debía cumplir el proyecto.

Luego se realizó la comprensión de los datos mediante un análisis exploratorio, donde se revisó la estructura del conjunto de datos, los tipos de variables, la existencia de valores nulos, la distribución de los datos y el desbalance entre las clases.

En la etapa de preparación de los datos se realizaron las transformaciones necesarias antes del entrenamiento de los modelos. Para ello se eliminaron variables que no aportaban al análisis, se aplicó One-Hot Encoding a las variables categóricas, se estandarizaron las variables numéricas y se utilizó SMOTE para balancear la clase minoritaria.

Posteriormente, se entrenaron tres modelos de aprendizaje automático: Random Forest, XGBoost y Perceptrón Multicapa (MLP). Una vez entrenados, su desempeño fue comparado mediante las métricas Accuracy, Precision, Recall, F1-Score y AUC-ROC, además de la matriz de confusión y la curva ROC.

Finalmente, se seleccionó XGBoost por obtener los mejores resultados durante la evaluación. Sobre este modelo se aplicó la técnica SHAP, con el objetivo de identificar cuáles fueron las variables que tuvieron mayor influencia en las predicciones realizadas.



6. Dinámica de Trabajo y Planificación

Para llevar a cabo el desarrollo de este proyecto de manera ordenada, colaborativa y con metas claras, hemos estructurado nuestra hoja de ruta global contemplando las tres evaluaciones sumativas del curso. Esta planificación general abarca desde la actual fase formativa hasta la consolidación final del proyecto, reflejando cómo el equipo distribuye responsabilidades según las fortalezas de cada integrante.

Nuestro plan de trabajo integral se divide en las siguientes etapas clave:

6.1. Proyecto Sumativa 1: Fundamentos y Modelo Predictivo (17 de junio al 28 de julio del 2026)

Esta primera gran etapa da vida al núcleo de nuestra plataforma *Perfectime* y se desglosa en cuatro fases de trabajo semanal:

Fase 1: Comprensión del Entorno (17 al 22 de junio): Arrancamos enfocándonos en el problema real. Integradoras miradas para analizar el impacto operacional de las paradas de máquina, definir KPIs críticos y consolidar el marco industrial.

Fase 2: El “Artesanado” de los Datos (23 de junio al 6 de julio): Con la convicción de que un buen modelo nace de datos limpios, se realiza el preprocesamiento con *pandas* y *matplotlib*, se mitió el ruido y el desbalanceo de clases y se identificó los patrones previos a las fallas.

Fase 3: Entrenamiento y Experimentación (7 al 20 de julio): Llevamos los datos al modelado, se realizó la configuración de algoritmos (Random Fores, SVM y XGBoost), mientras se evaluaba el desempeño con el F1-Score y se cerraba con la optimización y validación.

Fase 4: Transparencia y Explicabilidad (21 al 28 de julio): Evitamos el efecto de “caja negra! Integrando técnicas como SHAP para desentrañar el razonamiento del modelo y traducirlo en alertas operativas claras.

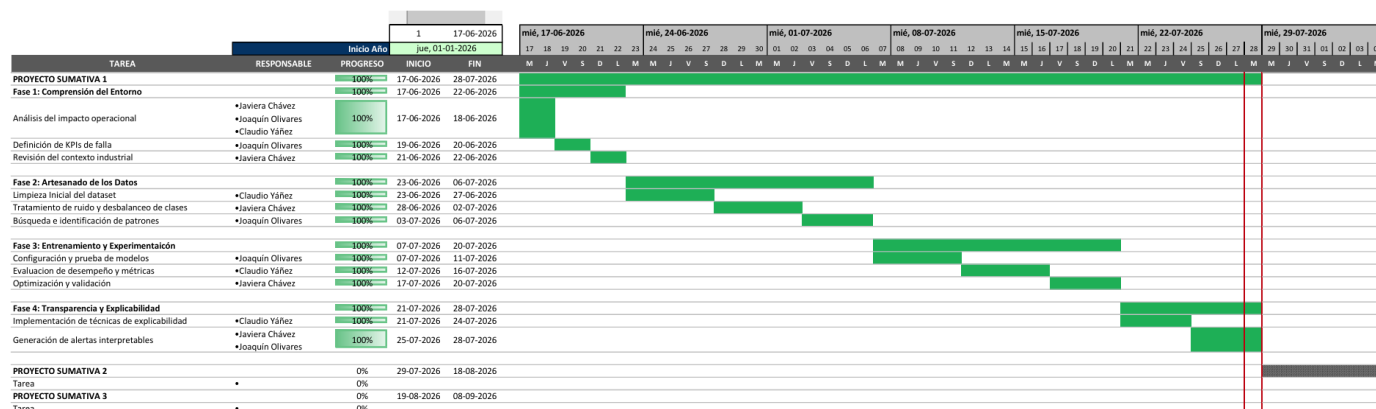


Figura 1. Carta Gantt del proyecto

7. Desarrollo experimental.

El proceso consideró el análisis exploratorio y la evaluación de la calidad de los datos, el preprocesamiento de las variables, la preparación del conjunto de entrenamiento, el desarrollo de distintos modelos de clasificación y la interpretación de sus resultados. Estas etapas permitieron identificar el modelo con mejor desempeño para

apoyar la predicción de fallas en equipos industriales, manteniendo coherencia con el objetivo general del proyecto y con los antecedentes revisados en la literatura.

7.1. Análisis Exploratorio de los datos

El análisis exploratorio de datos (EDA) constituyó la primera etapa del desarrollo experimental y tuvo como propósito comprender las características del conjunto de datos antes de iniciar el proceso de entrenamiento de los modelos. Esta fase permitió conocer la estructura del dataset, identificar el comportamiento de las variables y detectar posibles problemas que pudieran afectar el rendimiento de los algoritmos de aprendizaje automático.

Esta etapa es especialmente relevante en proyectos de mantenimiento predictivo, ya que la calidad y comprensión de los datos influyen directamente en la capacidad del modelo para identificar patrones asociados a fallas. Diversos autores destacan que una adecuada exploración de los datos permite detectar inconsistencias y comprender mejor las relaciones entre las variables antes de construir modelos predictivos, contribuyendo a obtener resultados más confiables.

7.1.1. Descripción del conjunto de datos.

Como primera actividad se verificó la correcta carga del conjunto de datos AI4I 2020 Predictive Maintenance Dataset, utilizado como base para el desarrollo del proyecto. Este dataset contiene 10.000 registros correspondientes al funcionamiento de equipos industriales e incorpora variables operacionales obtenidas mediante sensores, tales como temperatura del aire, temperatura del proceso, velocidad de rotación, torque y desgaste de la herramienta, además de una variable objetivo que indica si ocurrió o no una falla en la máquina.

La revisión inicial permitió comprobar que la información se encontraba correctamente disponible para su análisis y que las variables incluidas representan las condiciones de operación necesarias para desarrollar un modelo de mantenimiento predictivo, en concordancia con la problemática planteada en este proyecto.

Número de filas: 10000
Número de columnas: 14

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

7.1.2. Información general del conjunto de datos.

Posteriormente se utilizó la función `info()` de Pandas para revisar la estructura general del dataset. Este análisis permitió identificar el número de registros, la cantidad de variables, el tipo de dato de cada una y la existencia de valores no nulos.

Los resultados muestran que el conjunto de datos está compuesto por variables numéricas y categóricas, sin evidencia de registros incompletos. Además, el reducido tamaño del dataset facilita su procesamiento durante las etapas posteriores de preprocesamiento y entrenamiento.

```

]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   UDI                    10000 non-null  int64
1   Product ID            10000 non-null  str
2   Type                  10000 non-null  str
3   Air temperature [K]    10000 non-null  float64
4   Process temperature [K] 10000 non-null  float64
5   Rotational speed [rpm] 10000 non-null  int64
6   Torque [Nm]           10000 non-null  float64
7   Tool wear [min]        10000 non-null  int64
8   Machine failure        10000 non-null  int64
9   TWF                   10000 non-null  int64
10  HDF                   10000 non-null  int64
11  PWF                   10000 non-null  int64
12  OSF                   10000 non-null  int64
13  RNF                   10000 non-null  int64
dtypes: float64(3), int64(9), str(2)
memory usage: 1.1 MB

```

7.1.3. Estadísticas descriptivas

Con el objetivo de obtener una visión general del comportamiento de las variables numéricas, se calcularon estadísticas descriptivas como la media, desviación estándar, valores mínimos, máximos y cuartiles.

Estas medidas permitieron conocer los rangos de operación de cada sensor y verificar que los valores observados son consistentes con el contexto del problema. Esta información resulta útil para interpretar posteriormente los resultados obtenidos por los modelos y detectar posibles valores atípicos.

	count	mean	std	min	25%	50%	75%	max
UDI	10000.0	5000.50000	2886.895680	1.0	2500.75	5000.5	7500.25	10000.0
Air temperature [K]	10000.0	300.00493	2.000259	295.3	298.30	300.1	301.50	304.5
Process temperature [K]	10000.0	310.00556	1.483734	305.7	308.80	310.1	311.10	313.8
Rotational speed [rpm]	10000.0	1538.77610	179.284096	1168.0	1423.00	1503.0	1612.00	2886.0
Torque [Nm]	10000.0	39.98691	9.968934	3.8	33.20	40.1	46.80	76.6
Tool wear [min]	10000.0	107.95100	63.654147	0.0	53.00	108.0	162.00	253.0
Machine failure	10000.0	0.03390	0.180981	0.0	0.00	0.0	0.00	1.0
TWF	10000.0	0.00460	0.067671	0.0	0.00	0.0	0.00	1.0
HDF	10000.0	0.01150	0.106625	0.0	0.00	0.0	0.00	1.0
PWF	10000.0	0.00950	0.097009	0.0	0.00	0.0	0.00	1.0
OSF	10000.0	0.00980	0.098514	0.0	0.00	0.0	0.00	1.0
RNF	10000.0	0.00190	0.043550	0.0	0.00	0.0	0.00	1.0

7.1.4. Completitud de los datos.

Como parte del análisis exploratorio se verificó la existencia de valores nulos y registros duplicados. Esta revisión es importante porque datos incompletos o repetidos pueden introducir sesgos durante el entrenamiento de

los modelos y afectar la calidad de las predicciones.

Los resultados muestran que el dataset no presenta valores faltantes ni registros duplicados, por lo que fue posible continuar con las siguientes etapas sin necesidad de aplicar técnicas de imputación o eliminación de registros.

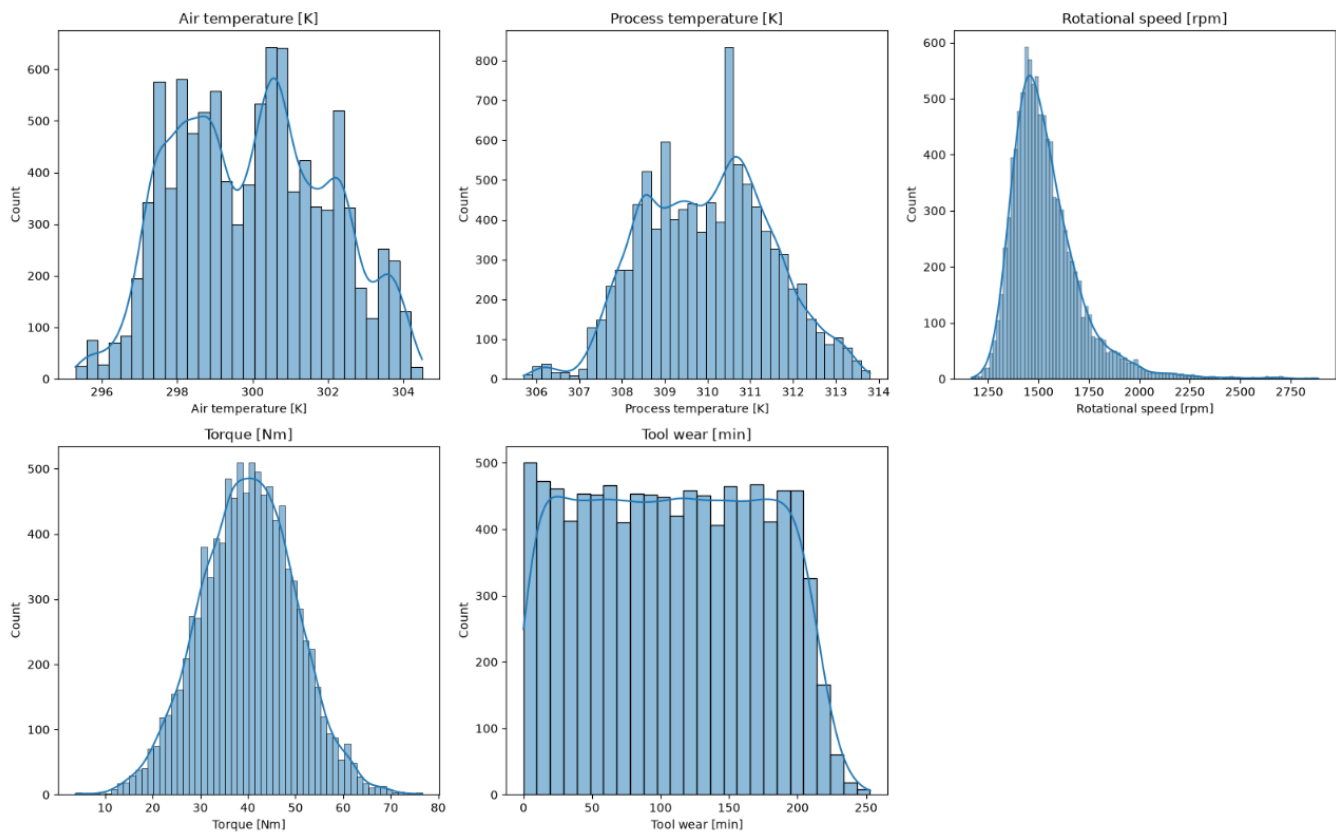
	Valores nulos	Porcentaje (%)
UDI	0	0.0
Product ID	0	0.0
Type	0	0.0
Air temperature [K]	0	0.0
Process temperature [K]	0	0.0
Rotational speed [rpm]	0	0.0
Torque [Nm]	0	0.0
Tool wear [min]	0	0.0
Machine failure	0	0.0
TWF	0	0.0
HDF	0	0.0
PWF	0	0.0
OSF	0	0.0
RNF	0	0.0

Registros duplicados: 0

7.1.5. Distribución de las variables.

Se analizaron las distribuciones de las variables numéricas mediante histogramas con el propósito de observar la forma en que se distribuyen los datos y detectar posibles comportamientos anómalos.

Se observó que algunas variables presentan distribuciones cercanas a una distribución normal, mientras que otras muestran cierto grado de asimetría, comportamiento esperable considerando que representan distintas condiciones de operación de equipos industriales. Este análisis permitió comprender mejor la naturaleza de los datos antes de aplicar las etapas de preprocesamiento.



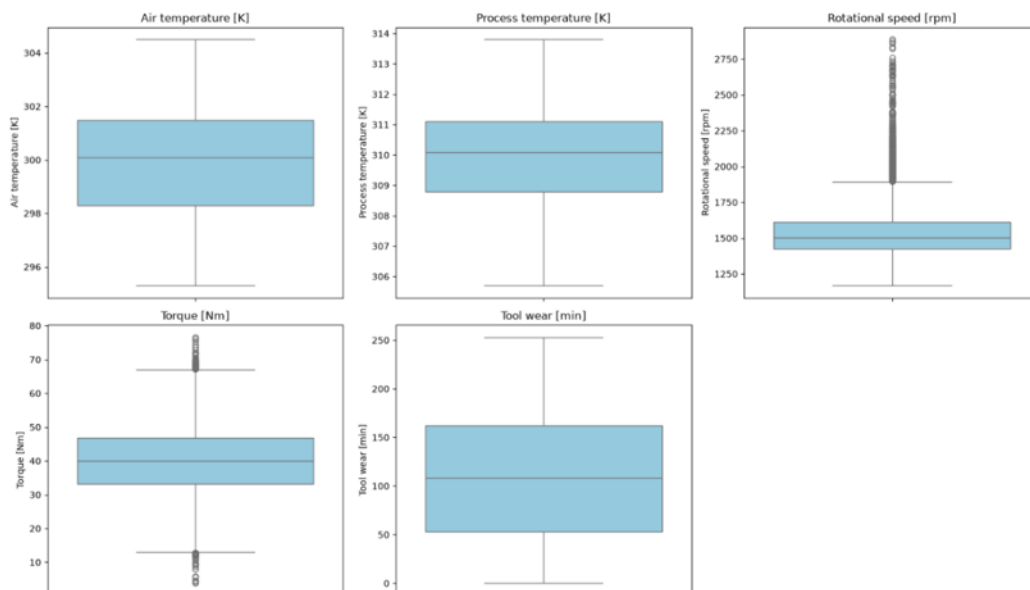
7.1.6. Identificación de valores atípicos.

Con el objetivo de identificar posibles valores atípicos (outliers), se construyeron diagramas de caja (boxplots) para cada una de las variables numéricas. Este tipo de visualización permite detectar observaciones que se encuentran alejadas del comportamiento general de los datos y evaluar si podrían afectar las etapas posteriores del análisis.

Los resultados muestran la presencia de algunos valores extremos, principalmente en variables como la velocidad de rotación, el torque y el desgaste de la herramienta. Sin embargo, en el contexto del mantenimiento predictivo estos registros no necesariamente corresponden a errores de medición o captura de datos. Por el contrario, es posible que representen condiciones reales de operación de una máquina sometida a altas cargas de trabajo, desgaste avanzado o situaciones cercanas a una falla.

Por esta razón se decidió no eliminar los valores atípicos durante el preprocesamiento. Su eliminación podría reducir la capacidad de los modelos para aprender patrones asociados a eventos poco frecuentes, precisamente aquellos que se busca identificar mediante el mantenimiento predictivo. Esta decisión también es consistente con los trabajos revisados, los cuales destacan que las variables registradas por sensores contienen información relevante sobre el comportamiento de los equipos y que el desafío principal consiste en detectar oportunamente las condiciones que anteceden a una falla.

Detección de valores atípicos en las variables numéricas

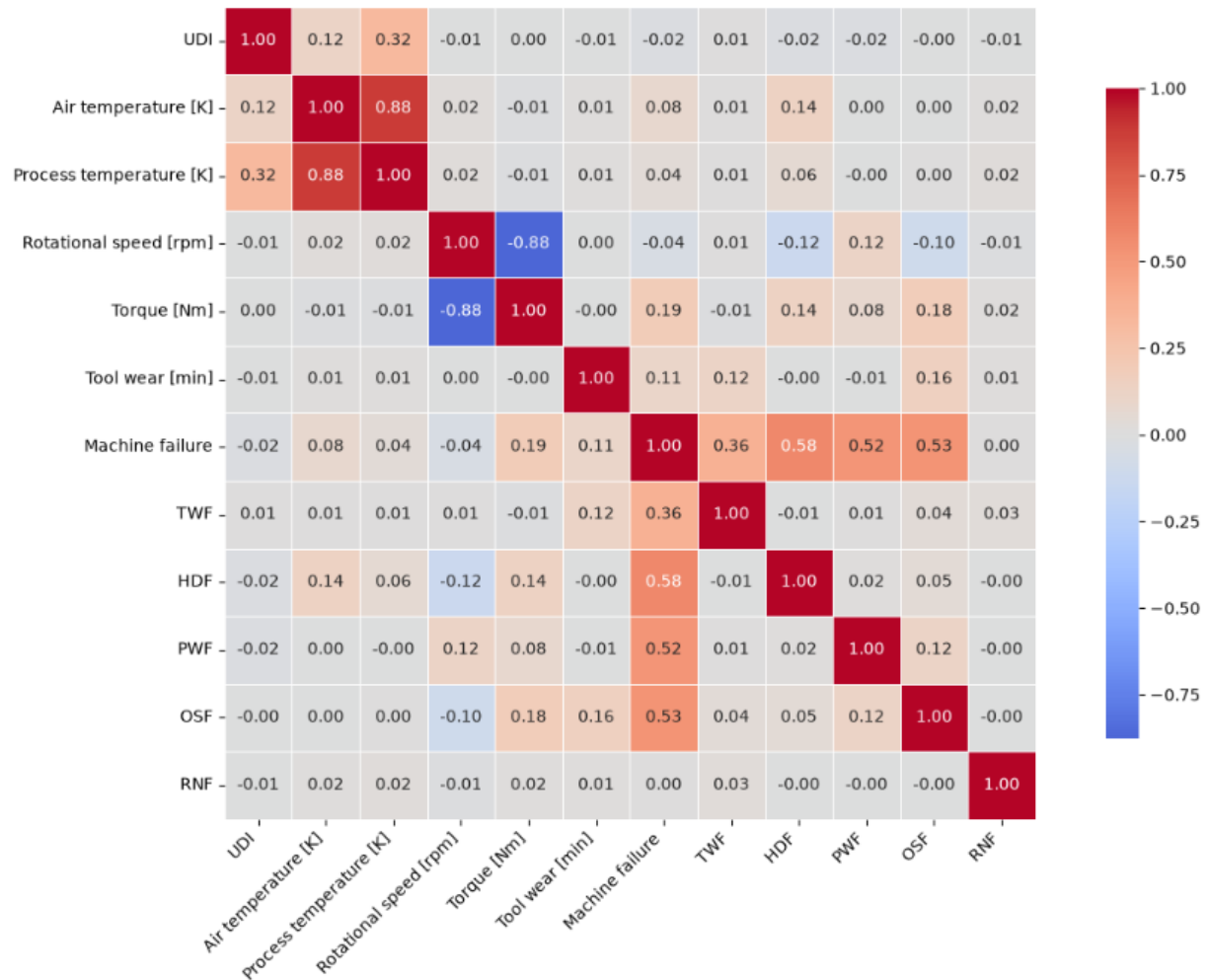


7.1.7. Análisis de Correlación.

Finalmente, se construyó una matriz de correlación utilizando el coeficiente de Pearson para analizar la relación existente entre las variables numéricas.

Los resultados evidencian una correlación positiva entre la temperatura del aire y la temperatura del proceso, mientras que variables como la velocidad de rotación y el torque presentan una relación inversa. En general, las demás variables muestran correlaciones bajas o moderadas, lo que indica que cada una aporta información complementaria al proceso de predicción. Este comportamiento resulta favorable para el entrenamiento de los modelos, ya que disminuye el riesgo de incorporar variables altamente redundantes.

Matriz de correlación entre las variables del dataset



7.2. Calidad de los datos.

Una vez realizado el análisis exploratorio, se evaluó la calidad del conjunto de datos con el objetivo de verificar que la información fuera adecuada para el entrenamiento de los modelos de aprendizaje automático. Para ello se analizaron distintas dimensiones de calidad, incluyendo la completitud, consistencia, exactitud y el balance de clases. Esta evaluación permitió identificar posibles problemas que pudieran afectar el desempeño de los modelos y definir las acciones necesarias antes de la etapa de preprocesamiento.

7.2.1. Completitud.

La dimensión de completitud evalúa si el conjunto de datos presenta información faltante que pueda afectar el análisis posterior. Para ello se verificó la existencia de valores nulos en cada una de las variables del dataset.

Los resultados muestran que todas las variables poseen la totalidad de sus registros completos, por lo que no fue necesario aplicar técnicas de imputación ni eliminar observaciones. Esto permitió continuar con las siguientes etapas del proyecto utilizando la totalidad de la información disponible.

7.2.2. Consistencia.

La consistencia hace referencia a que los datos mantengan un formato uniforme y coherente entre todas las

observaciones. En esta etapa se comprobó la existencia de registros duplicados y se revisó que las variables presentaran tipos de datos compatibles con su naturaleza.

El análisis confirmó que el conjunto de datos no presenta registros duplicados y que cada variable posee un tipo de dato adecuado para su procesamiento. Esto reduce la posibilidad de introducir errores durante las etapas de transformación y entrenamiento de los modelos.

		Mínimo	Máximo
Tipos de datos			
UDI	int64		
Product ID	str		
Type	str		
Air temperature [K]	float64		
Process temperature [K]	float64		
Rotational speed [rpm]	int64		
Torque [Nm]	float64		
Tool wear [min]	int64		
Machine failure	int64		
TWF	int64		
HDF	int64		
PWF	int64		
OSF	int64		
RNF	int64		
dttype: object			
Rangos de las variables numéricas			
		UDI	1.0 10000.0
		Air temperature [K]	295.3 304.5
		Process temperature [K]	305.7 313.8
		Rotational speed [rpm]	1168.0 2886.0
		Torque [Nm]	3.8 76.6
		Tool wear [min]	0.0 253.0
		Machine failure	0.0 1.0
		TWF	0.0 1.0
		HDF	0.0 1.0
		PWF	0.0 1.0
		OSF	0.0 1.0
		RNF	0.0 1.0

7.2.3. Exactitud.

La exactitud se evaluó revisando que los valores registrados fueran coherentes con el comportamiento esperado de las variables operacionales. Para ello se utilizaron las estadísticas descriptivas y el análisis realizado durante la exploración de los datos.

Si bien se identificaron algunos valores extremos, estos corresponden a condiciones plausibles de funcionamiento de los equipos industriales y no evidencian errores de captura o medición. En consecuencia, se consideró que el conjunto de datos mantiene un nivel adecuado de exactitud para el desarrollo del proyecto.

```
Verificación de valores negativos
Air temperature [K]: 0
Process temperature [K]: 0
Rotational speed [rpm]: 0
Torque [Nm]: 0
Tool wear [min]: 0
```

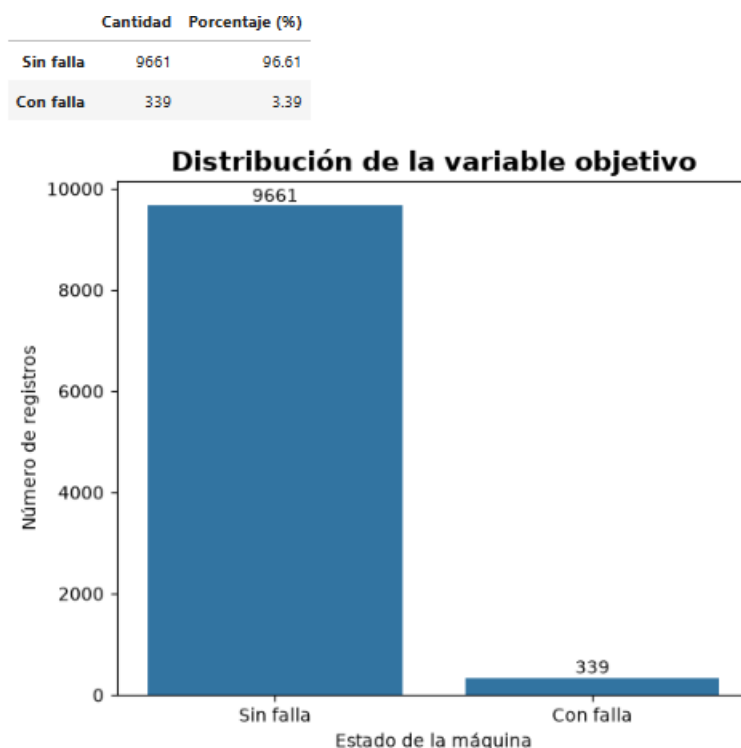
```
Categorías presentes en la variable Type
```

Categoría	
0	H
1	L
2	M

7.2.4. Balance de clases.

Finalmente, se analizó la distribución de la variable objetivo Machine Failure, observándose un marcado desbalance entre las clases. La mayoría de los registros corresponde a máquinas sin falla, mientras que los casos de falla representan una proporción considerablemente menor del conjunto de datos.

Este comportamiento es habitual en problemas de mantenimiento predictivo, ya que las fallas suelen ocurrir con mucha menor frecuencia que las condiciones normales de operación. Sin embargo, este desbalance puede afectar el entrenamiento de los modelos, favoreciendo la clase mayoritaria y disminuyendo la capacidad para detectar fallas reales. Por esta razón, en la etapa de preparación de los datos se decidió aplicar una técnica de balanceo basada en SMOTE, la cual será descrita en el apartado 6.4.



7.3. Preprocesamiento de los datos.

Una vez verificada la calidad del conjunto de datos, se realizó el preprocesamiento con el objetivo de preparar la información para las etapas posteriores de entrenamiento. En esta fase se aplicaron distintas transformaciones orientadas a eliminar variables que no aportaban información relevante al modelo y adaptar el formato de los datos para que pudiera ser procesado correctamente por los algoritmos de aprendizaje automático.

Estas transformaciones permiten reducir información redundante, mejorar la calidad del conjunto de datos y asegurar que todas las variables sean interpretadas correctamente por los modelos, contribuyendo a obtener resultados más consistentes durante el proceso de entrenamiento.

7.3.1. Eliminación de variables no predictivas.

Como primera etapa se eliminaron las variables UDI y Product ID, ya que corresponden únicamente a un identificador único de cada registro y al código del producto, respectivamente. Estas variables no describen el comportamiento operativo de la máquina y, por lo tanto, no aportan información útil para predecir la ocurrencia de fallas.

Su eliminación permite evitar que el modelo aprenda relaciones que no tienen significado desde el punto de vista del problema y reduce la cantidad de información innecesaria durante el entrenamiento.

Columnas del dataset después del preprocesamiento:

Columnas conservadas	
0	Type
1	Air temperature [K]
2	Process temperature [K]
3	Rotational speed [rpm]
4	Torque [Nm]
5	Tool wear [min]
6	Machine failure

7.3.2. Normalización de nombres de variables.

Posteriormente se realizó la normalización de los nombres de las columnas, reemplazando espacios y caracteres especiales por un formato uniforme basado en letras minúsculas y guiones bajos (snake_case).

Aunque esta transformación no modifica el contenido de los datos, facilita la manipulación del dataset durante el desarrollo del proyecto, mejora la legibilidad del código y reduce la posibilidad de errores al momento de acceder a las variables desde Python.

Nombres de columnas normalizados:

Columnas	
0	Type
1	Air_temperature_K
2	Process_temperature_K
3	Rotational_speed_rpm
4	Torque_Nm
5	Tool_wear_min
6	Machine_failure

7.3.3. Codificación de variables categóricas.

Finalmente, la variable categórica Type fue transformada mediante la técnica **One-Hot Encoding**, generando nuevas variables binarias que representan cada una de sus categorías.

Esta transformación fue necesaria porque los algoritmos utilizados en este proyecto trabajan con datos numéricos y no pueden interpretar directamente variables de tipo texto. Mediante One-Hot Encoding se evita introducir un orden artificial entre las categorías y se conserva la información original de la variable para que pueda ser utilizada durante el entrenamiento.

Una vez aplicada esta transformación, el conjunto de datos quedó completamente preparado para separar las variables predictoras de la variable objetivo.

Columnas después de la codificación:

Columnas	
0	Air_temperature_K
1	Process_temperature_K
2	Rotational_speed_rpm
3	Torque_Nm
4	Tool_wear_min
5	Machine_failure
6	Type_L
7	Type_M

7.3.4. Separación de variables predictoras y variable objetivo.

Como último paso del preprocesamiento se separó la variable objetivo Machine Failure del resto de las variables predictoras. De esta forma se definió la matriz de características (X) y el vector objetivo (y), estructura necesaria para continuar con la división de los datos en conjuntos de entrenamiento y prueba.

Esta separación marca el término del preprocesamiento y da inicio a la etapa de preparación de los datos para el entrenamiento de los modelos de clasificación.

Número de variables predictoras: 7
Número de registros: 10000

Variable objetivo:

Cantidad	
Machine_failure	
0	9661
1	339

7.4. Preparación para el entrenamiento.

Una vez finalizado el preprocesamiento, se preparó el conjunto de datos para el entrenamiento de los modelos de clasificación. En esta etapa se realizaron tres procedimientos fundamentales: la división de los datos en conjuntos de entrenamiento y prueba, el escalamiento de las variables numéricas y el balanceo de las clases mediante la técnica SMOTE. Estas acciones permitieron generar un conjunto de datos más adecuado para el aprendizaje de los algoritmos y mejorar su capacidad de generalización.

7.4.1. División de los datos.

Como primera etapa, el conjunto de datos fue dividido en dos subconjuntos utilizando la función `train_test_split()` de Scikit-learn. Se destinó un 70 % de los registros para entrenamiento y el 30 % restante para pruebas, permitiendo reservar un conjunto independiente para evaluar posteriormente el desempeño de los modelos.

Esta separación ayuda a verificar la capacidad de generalización de los algoritmos, ya que las métricas finales se obtienen utilizando información que no fue empleada durante el entrenamiento.

7.4.2. Escalado de las variables.

Posteriormente, las variables numéricas fueron estandarizadas mediante la técnica `StandardScaler`, la cual

transforma los datos para que presenten una escala comparable entre sí.

Este procedimiento resulta especialmente útil para algoritmos sensibles a la magnitud de las variables, como las redes neuronales, ya que favorece un proceso de entrenamiento más estable y evita que variables con valores de mayor escala tengan una influencia desproporcionada sobre el modelo.

Primeros registros del conjunto de entrenamiento escalado:

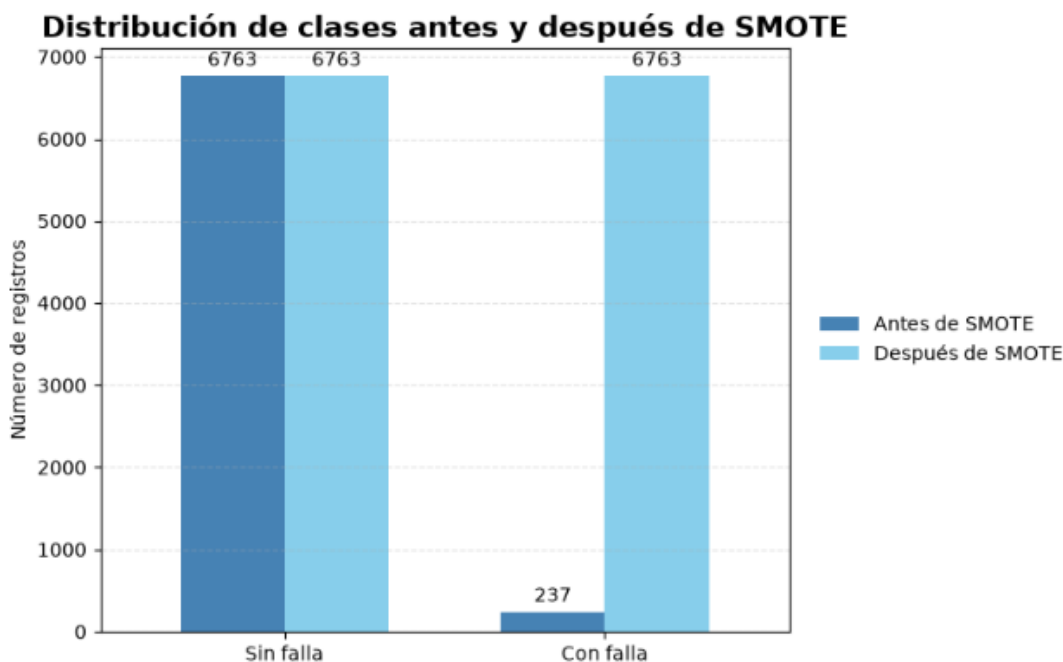
	Air_temperature_K	Process_temperature_K	Rotational_speed_rpm	Torque_Nm	Tool_wear_min	Type_L	Type_M
0	-1.105779	-1.759415	2.012835	-1.569298	0.334742	-1.230964	1.539033
1	1.840492	1.547973	-1.055557	1.099287	-0.041084	0.812372	-0.649758
2	-1.405400	-1.421926	-0.256554	-0.160045	0.600952	0.812372	-0.649758
3	1.790556	1.547973	0.503608	-0.759727	1.806727	-1.230964	1.539033
4	0.442262	1.277982	3.333409	-2.348885	-1.011967	-1.230964	-0.649758

7.4.3. Balanceo de clases mediante SMOTE.

Finalmente, se aplicó la técnica SMOTE (Synthetic Minority Over-sampling Technique) sobre el conjunto de entrenamiento con el propósito de corregir el desbalance existente en la variable objetivo.

SMOTE genera nuevos ejemplos sintéticos para la clase minoritaria a partir de observaciones similares, permitiendo equilibrar la distribución de las clases sin limitarse a duplicar registros existentes. De esta forma, los modelos disponen de una representación más equilibrada durante el entrenamiento, mejorando su capacidad para identificar tanto los casos normales como las situaciones de falla.

Es importante destacar que esta técnica fue aplicada únicamente sobre el conjunto de entrenamiento, evitando incorporar información del conjunto de prueba durante el proceso de aprendizaje y asegurando una evaluación objetiva del desempeño de los modelos.



7.5. Entrenamiento de los modelos

Una vez preparados los datos, se procedió al entrenamiento de los modelos de aprendizaje automático seleccionados para el proyecto. En esta etapa se utilizaron los datos de entrenamiento previamente procesados para ajustar cada algoritmo y generar modelos capaces de predecir la ocurrencia de fallas en las

máquinas.

Con el propósito de comparar su desempeño, se entrenaron tres algoritmos supervisados: Random Forest, XGBoost y una Red Neuronal Multicapa (MLP). Cada uno fue implementado utilizando las configuraciones definidas en el notebook y posteriormente evaluado mediante distintas métricas de clasificación.

- **Modelo Random Forest:** El primer modelo entrenado fue Random Forest, un algoritmo basado en un conjunto de árboles de decisión que combina las predicciones de múltiples árboles para obtener una clasificación final. Para el entrenamiento se utilizó la implementación RandomForestClassifier de Scikit-learn, configurando 100 árboles de decisión (`n_estimators = 100`) y un `random_state = 42` para asegurar la reproducibilidad de los resultados. Posteriormente, el modelo fue ajustado utilizando el conjunto de entrenamiento balanceado mediante SMOTE y, una vez finalizado el entrenamiento, se realizaron las predicciones sobre el conjunto de prueba
- **Modelo XGBoost:** Posteriormente, se entrenó el modelo XGBoost, un algoritmo basado en la técnica de Gradient Boosting, el cual construye de forma secuencial un conjunto de árboles de decisión con el objetivo de mejorar progresivamente el rendimiento del modelo. Para este proyecto se utilizó la clase XGBClassifier, configurando `random_state = 42` y la métrica de evaluación `logloss` mediante el parámetro `eval_metric`. Al igual que el modelo anterior, el entrenamiento se realizó utilizando el conjunto de datos balanceado y posteriormente se generaron las predicciones sobre el conjunto de prueba para su evaluación.
- **Modelo Perceptrón multicapa:** Finalmente, se entrenó un modelo de Perceptrón Multicapa (MLP) utilizando la implementación MLPClassifier de Scikit-learn. La arquitectura de la red neuronal se configuró con tres capas ocultas de 64, 32 y 16 neuronas, utilizando la función de activación ReLU, el optimizador Adam y un máximo de 500 iteraciones para el proceso de entrenamiento. Además, se estableció `random_state = 42` con el fin de garantizar la reproducibilidad de los resultados. Una vez entrenada la red neuronal con el conjunto de entrenamiento balanceado, el modelo generó las predicciones correspondientes sobre el conjunto de prueba para su posterior evaluación.

7.6. Evaluación de modelos.

Una vez entrenados los modelos, se evaluó su desempeño utilizando el conjunto de prueba, el cual no participó durante el entrenamiento ni en el proceso de balanceo mediante SMOTE. Esta evaluación permitió medir la capacidad de cada algoritmo para clasificar correctamente el estado de las máquinas y comparar objetivamente sus resultados.

Para ello se utilizaron diferentes métricas de clasificación, entre ellas Accuracy, Precision, Recall, F1-Score y el Área Bajo la Curva ROC (AUC-ROC). En conjunto, estas métricas permiten analizar tanto el rendimiento general de los modelos como su capacidad para identificar correctamente las máquinas con falla.

7.6.1. Reportes de clasificación.

Como primera evaluación, se generó el reporte de clasificación para cada uno de los modelos entrenados utilizando la función `classification_report()` de Scikit-learn. Este reporte resume el desempeño obtenido para cada clase mediante las métricas de Precision, Recall y F1-Score, además de la métrica Accuracy y el valor de AUC-ROC, permitiendo realizar una primera comparación entre los algoritmos.

Los resultados muestran que los tres modelos alcanzaron un alto desempeño en la clasificación de la clase mayoritaria (máquinas sin falla). Sin embargo, las diferencias más importantes se observaron en la detección de la clase minoritaria, correspondiente a las máquinas con falla.

En este aspecto, XGBoost obtuvo el mejor equilibrio entre Precision, Recall y F1-Score, logrando identificar una mayor proporción de fallas respecto de los demás modelos. Por su parte, el Perceptrón Multicapa (MLP) presentó un rendimiento intermedio, mientras que Random Forest obtuvo los resultados más bajos para la clase minoritaria.

Random Forest					XGBoost					Perceptrón Multicapa (MLP)				
	precision	recall	f1-score	support		precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.99	0.97	0.98	2898	0	0.99	0.98	0.99	2898	0	0.99	0.97	0.98	2898
1	0.47	0.72	0.57	102	1	0.61	0.77	0.68	102	1	0.50	0.73	0.59	102
accuracy			0.96	3000	accuracy			0.98	3000	accuracy			0.97	3000
macro avg	0.73	0.84	0.77	3000	macro avg	0.80	0.88	0.83	3000	macro avg	0.75	0.85	0.79	3000
weighted avg	0.97	0.96	0.97	3000	weighted avg	0.98	0.98	0.98	3000	weighted avg	0.97	0.97	0.97	3000
AUC-ROC: 0.8435					AUC-ROC: 0.8785					AUC-ROC: 0.8500				

7.6.2. Comparación de métricas de desempeño.

Con el objetivo de facilitar la comparación entre los algoritmos, se elaboró una tabla que resume las principales métricas obtenidas por cada modelo: Accuracy, Precision, Recall, F1-Score y AUC-ROC.

Los resultados evidencian que XGBoost obtuvo el mejor desempeño general, alcanzando un Accuracy de 97,53 %, una Precision de 60,77 %, un Recall de 77,45 %, un F1-Score de 68,10 % y un AUC-ROC de 0,8785. Estas métricas indican que fue el algoritmo con mayor capacidad para detectar correctamente las máquinas con falla, manteniendo un buen equilibrio entre precisión y sensibilidad.

El Perceptrón Multicapa (MLP) obtuvo un rendimiento intermedio, superando ligeramente a Random Forest en Recall, F1-Score y AUC-ROC. En cambio, Random Forest, aunque presentó una alta exactitud general, obtuvo los valores más bajos en la mayoría de las métricas asociadas a la detección de la clase minoritaria.

Con base en esta comparación, XGBoost se posicionó como el modelo con mejor desempeño para las siguientes etapas del proyecto.

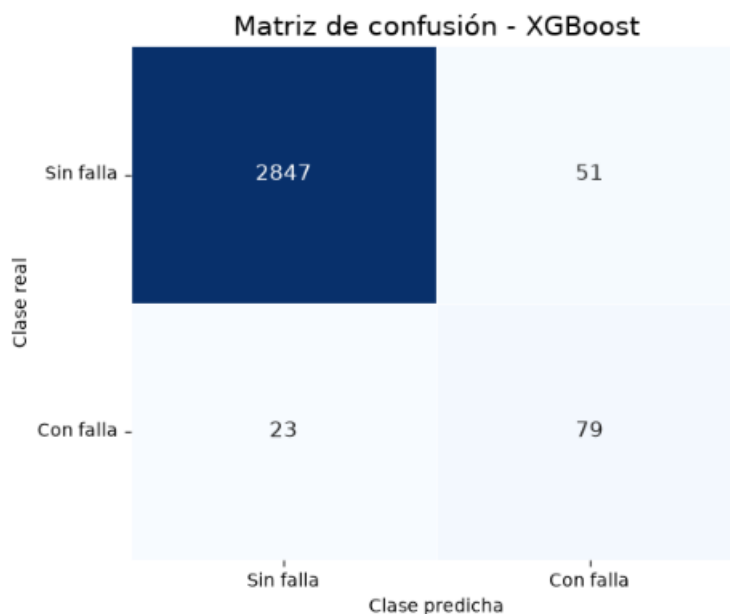
	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	0.9627	0.4679	0.7157	0.5659	0.8435
XGBoost	0.9753	0.6077	0.7745	0.6810	0.8785
Perceptrón Multicapa	0.9660	0.5000	0.7255	0.5920	0.8500

7.6.3. Matriz de confusión.

Con el propósito de analizar con mayor detalle el comportamiento del modelo seleccionado, se generó la matriz de confusión correspondiente a XGBoost. Esta herramienta permite visualizar la cantidad de predicciones correctas e incorrectas realizadas para cada una de las clases, diferenciando los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

Los resultados muestran que el modelo clasificó correctamente 2.847 máquinas sin falla y 79 máquinas con falla. Asimismo, registró 51 falsos positivos y 23 falsos negativos, lo que evidencia una buena capacidad para detectar la mayoría de las fallas presentes en el conjunto de prueba.

Esta representación complementa las métricas obtenidas anteriormente y facilita la interpretación del comportamiento del modelo frente a cada clase.

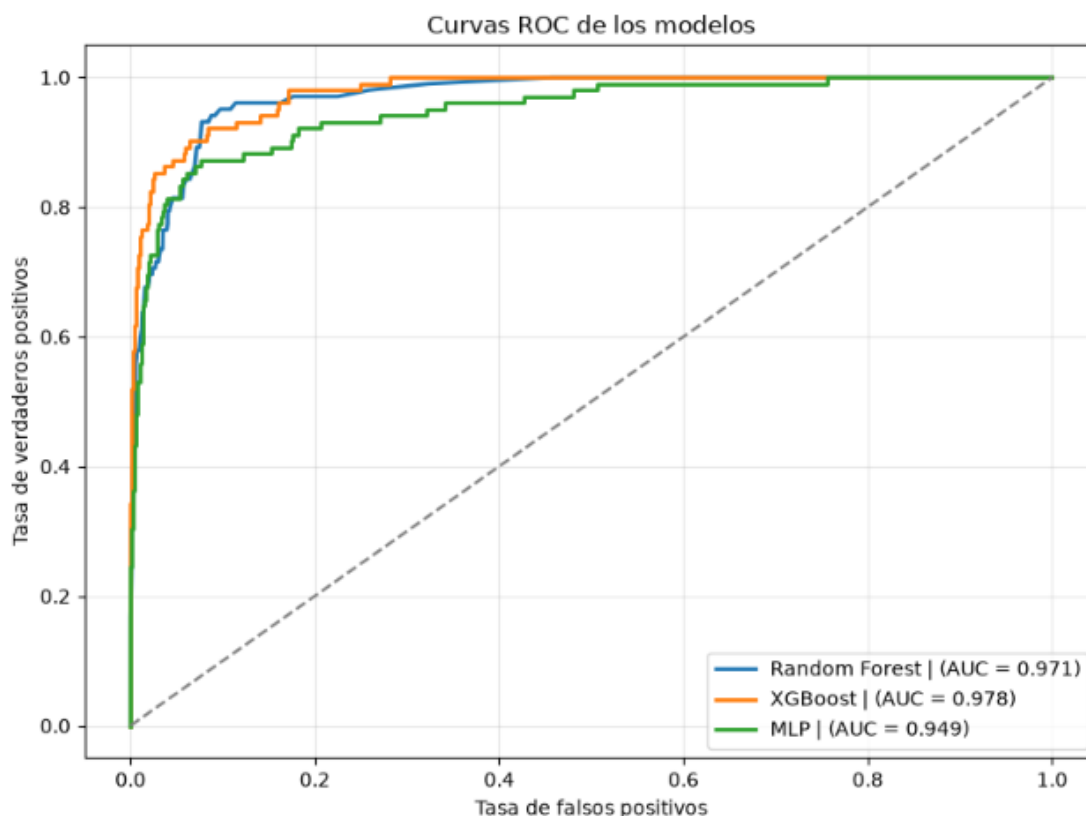


7.6.4. Curva ROC.

Como evaluación complementaria, se construyeron las curvas ROC para los tres modelos entrenados. Estas curvas permiten analizar la capacidad discriminativa de cada algoritmo al representar la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos para distintos umbrales de decisión.

Los resultados muestran que los tres modelos presentan un buen desempeño, ya que sus curvas se mantienen claramente por encima de la línea diagonal de referencia, la cual representa una clasificación aleatoria.

El modelo XGBoost obtuvo el mayor valor de AUC (0,978), seguido por Random Forest (0,971) y Perceptrón Multicapa (MLP) (0,949). Estos resultados confirman que XGBoost posee la mejor capacidad para diferenciar entre máquinas con falla y sin falla, respaldando las conclusiones obtenidas mediante las métricas de evaluación y la matriz de confusión.



7.6.5. Selección del mejor modelo.

A partir de los resultados obtenidos durante la evaluación, se seleccionó XGBoost como el modelo con mejor desempeño para este proyecto.

Esta decisión se fundamentó en que obtuvo los mejores resultados en las principales métricas de clasificación, incluyendo Accuracy, Precision, Recall, F1-Score y AUC-ROC, además de presentar el mejor comportamiento en la matriz de confusión para la detección de la clase minoritaria.

En consecuencia, XGBoost fue elegido para las etapas posteriores del proyecto, donde se aplicarán técnicas de interpretabilidad mediante SHAP con el fin de analizar la influencia de las variables en las predicciones realizadas por el modelo.

7.7. Interpretabilidad del modelo mediante SHAP.

Una vez seleccionado XGBoost como el modelo con mejor desempeño, se aplicó la técnica de interpretabilidad SHAP (SHapley Additive exPlanations) con el propósito de analizar la influencia de cada variable en las predicciones realizadas por el modelo.

SHAP permite estimar la contribución individual de cada característica sobre la predicción generada, facilitando la comprensión del comportamiento del modelo y entregando una explicación más transparente de las decisiones de clasificación.

7.7.1. Inicialización de SHAP.

Como primer paso, se inicializó la librería SHAP y se creó el explicador correspondiente al modelo XGBoost

mediante TreeExplainer, el cual está diseñado para interpretar modelos basados en árboles de decisión.

Posteriormente, el conjunto de prueba escalado fue convertido nuevamente a un DataFrame, conservando los nombres originales de las variables. Esta conversión permite que las explicaciones y gráficos generados por SHAP identifiquen correctamente cada característica, facilitando su interpretación.

Esta preparación constituye un paso previo necesario para calcular la contribución individual de cada variable en las predicciones del modelo seleccionado.

Conjunto preparado para el análisis SHAP:

	Air_temperature_K	Process_temperature_K	Rotational_speed_rpm	Torque_Nm	Tool_wear_min	Type_L	Type_M
0	-0.007169	0.940494	-0.012415	-0.459886	0.193808	-1.230964	1.539033
1	-1.405400	-1.219433	0.769942	-1.189500	0.397380	-1.230964	1.539033
2	0.192578	-0.206967	0.808783	-0.769722	0.929800	-1.230964	1.539033
3	-0.156980	-0.544456	-0.461854	0.059838	1.164691	-1.230964	-0.649758
4	0.342388	0.738001	0.825429	-0.929637	0.835844	0.812372	-0.649758

7.7.2. Importancia global de las variables.

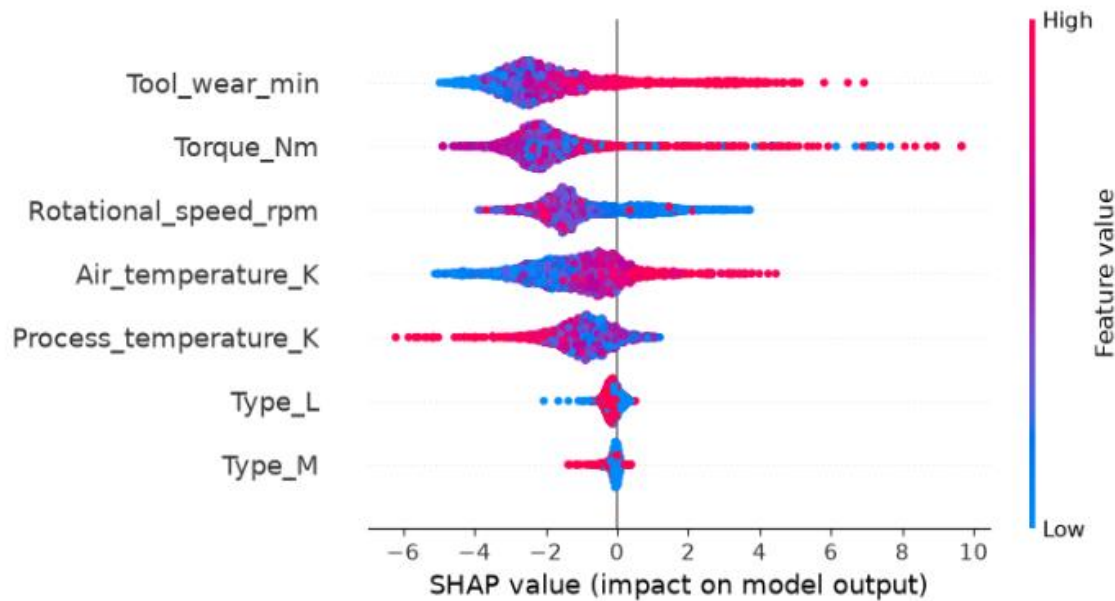
Una vez preparado el conjunto de datos, se calcularon los valores SHAP para todas las observaciones del conjunto de prueba y posteriormente se generó el gráfico SHAP Summary Plot, el cual resume la importancia global de las variables utilizadas por el modelo.

En este gráfico, las variables aparecen ordenadas según su influencia sobre las predicciones. Además, el color de cada punto representa el valor que toma la característica en una observación, mientras que su posición sobre el eje horizontal indica si dicha variable aumenta o disminuye la probabilidad de que el modelo prediga una falla.

Los resultados muestran que Tool_wear_min, Torque_Nm y Rotational_speed_rpm fueron las variables con mayor impacto en las predicciones realizadas por el modelo. En cambio, variables como Type_L y Type_M presentaron una menor contribución relativa dentro del proceso de clasificación.

Este análisis proporciona una visión global del comportamiento del modelo y permite identificar cuáles son las características más relevantes para la detección de fallas en las máquinas.

Calculando valores SHAP...
Generando gráfico resumen de SHAP...



8. Limitaciones del Modelo y Propuestas de Mejora

A pesar del sólido rendimiento alcanzado por las arquitecturas propuestas (destacando XGBoost y la exploración de modelos DL), la evaluación técnica del sistema *Perfectime* sobre el dataset, permite identificar limitaciones operacionales específicas que deben abordarse para garantizar su escalabilidad. A continuación, se detallan dichas restricciones y se fundamentan dos mejoras técnicas estratégicas para las siguientes fases del proyecto.

8.1. Identificación de Limitaciones del Modelo Actual

Naturaleza Estática de los Datos Tabulares: El conjunto de datos proporciona mediciones tomadas en instantes discretos de tiempo. Esta característica limita la capacidad de algoritmos como XGBoost para capturar la historia de degradación continua de un componente, omitiendo la evolución temporal previa a una falla por fatiga o desgaste.

Sensibilidad al Desbalanceo Severo de Clases: Aunque se aplican técnicas de rebalanceo (como SMOTE y ponderación `scale_pos_weight`), tipos de falla extremadamente infrecuentes como la Falla Aleatoria – RNF presentan un número tan reducido de muestras en el conjunto de entrenamiento que el modelo corre el riesgo de generar falsos negativos en escenarios críticos.

8.2. Propuestas de Mejora Fundamentadas

Mejora 1: Implementación de Ventanas Temporales Deslizantes (Time-Series Feature Engineering) y Arquitectura Híbrida CNN-LSTM

Fundamentación: Para superar la limitación de los datos estáticos, se propone transformar la ingesta de telemetría en ventanas secuenciales deslizantes (p. ej., agrupando los últimos minutos de operación). Sobre esta estructura secuencial se entrenará una arquitectura híbrida 1D-CNN + LSTM. La capa convolucional

extraerá patrones de fluctuación local en torque y temperatura, mientras que la celda recurrente LSTM modelará la acumulación paulatina de estrés mecánico. Esto permitirá estimar no solo si ocurrirá una falla, sino la Vida Útil Restante (RUL - Remaining Useful Life) del equipo.

Mejora 2: Integración de Aprendizaje Activo (Active Learning) y Monitoreo Metodológico de Deriva de Datos (Data Drift)

Fundamentación: En entornos industriales reales, las condiciones operativas (temperatura ambiente, carga de trabajo) cambian constantemente, provocando degradación en la precisión de las predicciones (*Concept Drift*). Se propone integrar un módulo de Aprendizaje Activo que identifique de forma automática las predicciones con menor margen de certidumbre (p. ej., probabilidades entre 0.45 y 0.55 en XGBoost). Estas instancias dudosas serán enviadas a la interfaz del supervisor para validación humana, retroalimentando y reentrenando periódicamente el modelo. Esta mejora reduce los falsos positivos y asegura la adaptabilidad continua del sistema a nuevas condiciones de planta.

9. Arquitectura Frontend/Backend

Se desarrolló un prototipo visual de alta fidelidad para representar cómo funcionaría el sistema de mantenimiento predictivo PERFECTIME. El objetivo de esta primera etapa es presentar la idea al cliente, mostrar la organización de las pantallas y validar la experiencia de uso antes de implementar el sistema completo.

Actualmente, la interfaz utiliza datos simulados para representar predicciones, alertas, gráficos, explicaciones SHAP, historial y métricas de monitoreo. En una etapa posteriores, estos elementos se conectarían con un backend, un modelo de Machine Learning y una base de datos.

9.1. Panel de control

El panel principal entrega una visión general del estado de las máquinas monitoreadas. Incluye indicadores como cantidad de máquinas, alertas activas, precisión estimada del modelo y archivos procesados. También presenta gráficos de tendencia, equipos con mayor riesgo y las últimas predicciones.

Esta pantalla permitiría que el supervisor identifique rápidamente las situaciones más importantes y priorice las acciones de mantenimiento. En el futuro, la información se actualizaría automáticamente mediante datos obtenidos desde el backend.

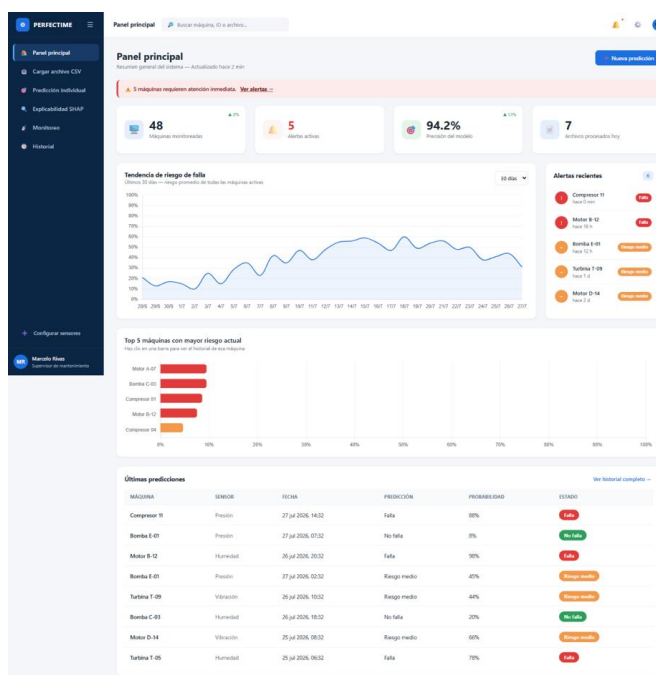


Figura 1. Panel principal de PERFECTIME. Se observa el sistema de navegación lateral, la jerarquía visual mediante tarjetas KPI y la paleta de colores industrial (azul, verde, naranja, rojo) aplicada de forma consistente.

9.2.Carga de archivo CSV

Esta sección representa el proceso de carga masiva de información proveniente de sensores. El usuario podría seleccionar o arrastrar un archivo CSV, revisar una vista previa de los datos y solicitar su análisis.

En una futura implementación, el backend validaría el formato y contenido del archivo, aplicaría el preprocesamiento correspondiente y enviaría los registros al modelo predictivo. Actualmente, la carga, validación y análisis se muestran de forma simulada.

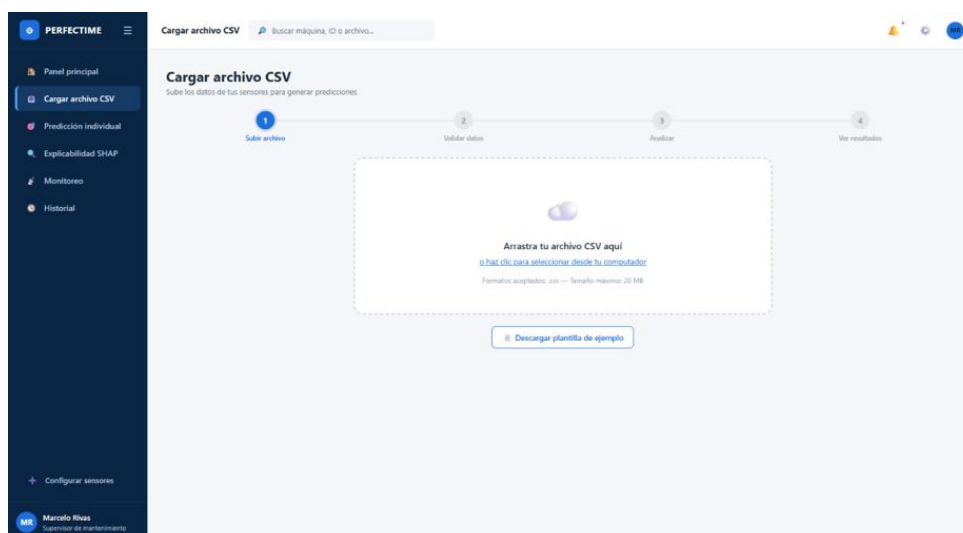


Figura 2. Estado inicial de la carga de datos: el área de arrastre y la plantilla descargable reducen la fricción y los errores de formato antes de subir el archivo.

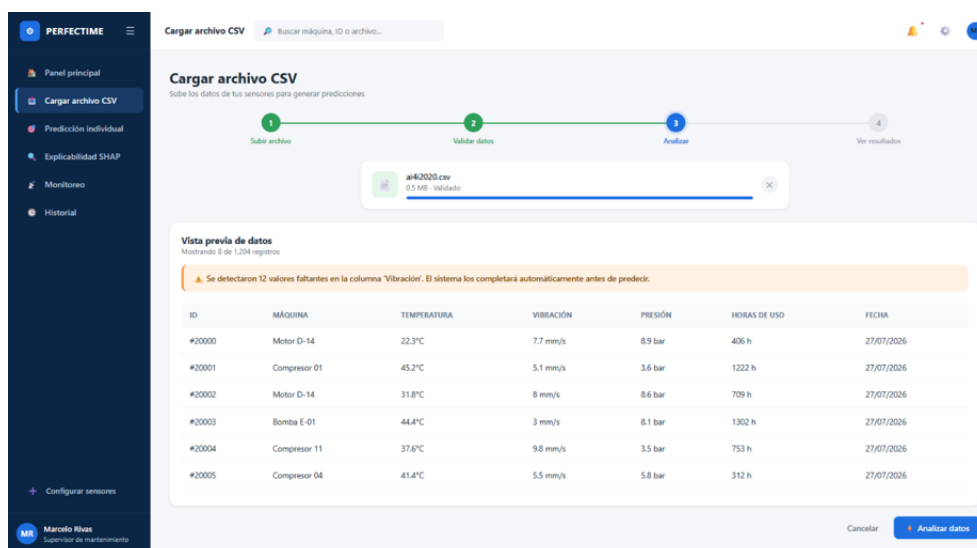


Figura 2.1 Vista previa de los datos cargados antes del análisis, incluyendo la detección automática de valores faltantes como mecanismos de transparencia hacia el usuario

9.3. Predicción individual

La pantalla de predicción individual permitiría evaluar una máquina específica mediante el ingreso manual de valores de sensores. El resultado se mostraría como una categoría de riesgo, acompañada de una probabilidad y de los factores más relevantes.

Esta funcionalidad facilitaría la evaluación rápida de casos puntuales sin necesidad de cargar un archivo completo. En una etapa posterior, los datos ingresados se enviarían al backend, donde serían procesados por el modelo de Machine Learning.

Figura 3. Formulario de predicción individual, con controles numéricos redundantes (campo + slider) para facilitar el ingreso de

datos en planta.

9.4. Explicabilidad SHAP

La sección de explicabilidad busca mostrar las razones que influyeron en una predicción. Para ello, presenta las variables que aumentan o reducen el riesgo, junto con un gráfico y una explicación en lenguaje comprensible.

Esta información permitiría que el usuario no reciba únicamente una clasificación, sino que también comprenda por qué se obtuvo ese resultado. En el sistema definitivo, el backend calcularía los valores SHAP reales utilizando el modelo entrenado.

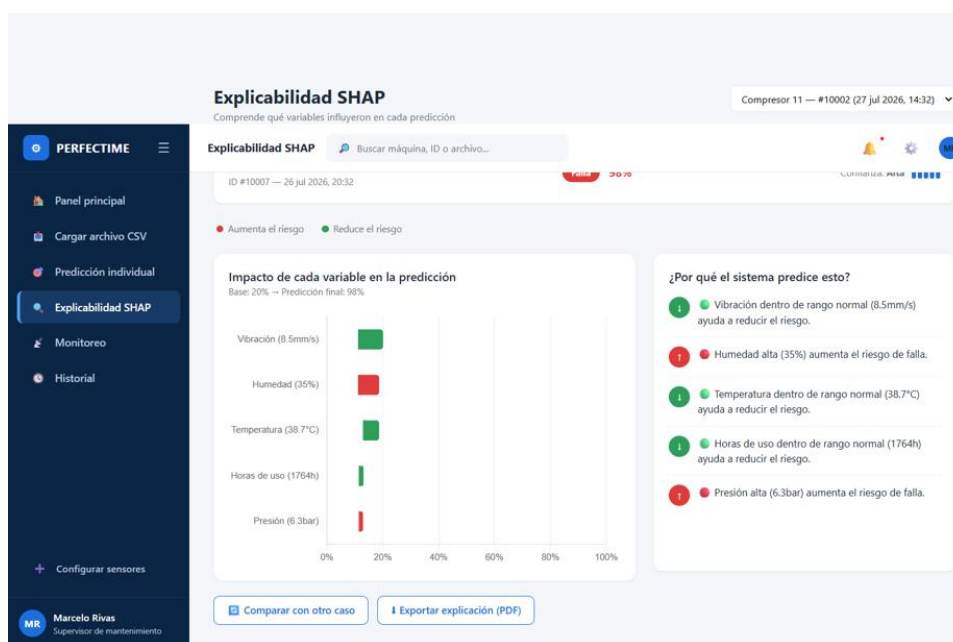


Figura 4. Gráfico SHAP tipo waterfall: cada barra representa el aporte individual de una variable al resultado final, coloreada según aumenta (rojo) o reduce (verde) el riesgo.

9.5. Monitoreo

La pantalla de monitoreo representa el seguimiento del rendimiento técnico del sistema. Incluye métricas como precisión del modelo, tiempo de respuesta, cantidad de análisis realizados, archivos procesados y alertas.

Su objetivo sería detectar posibles disminuciones en el desempeño del modelo o problemas operacionales. En una etapa futura, estas métricas se calcularían utilizando los registros reales generados durante el uso del sistema.

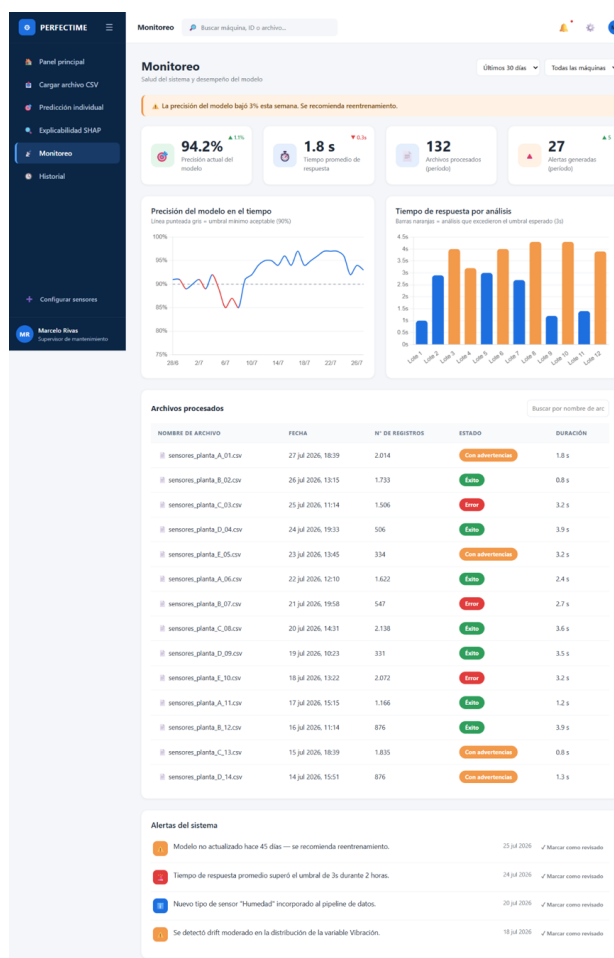


Figura 5. Panel de monitoreo del modelo: la caída de precisión por debajo del umbral del 90% se resalta automáticamente en el gráfico y mediante un banner de advertencia.

9.6. Historial

El historial permite visualizar las predicciones realizadas anteriormente. La interfaz incorpora búsqueda, filtros, ordenamiento, paginación y opciones de exportación.

Esta sección entregaría trazabilidad sobre las evaluaciones realizadas y facilitaría el seguimiento de una máquina a lo largo del tiempo. Posteriormente, los registros serían almacenados en una base de datos y consultados mediante el backend.

PERFECTIME

Historial
Consulta y filtra todas las predicciones realizadas

compresor No falla Tipo de máquina: Todos Origen: Todos Limpiar filtros

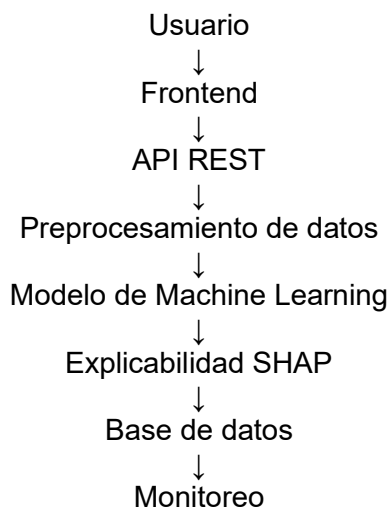
ID	MÁQUINA	FECHA TI	ORIGEN	PREDICCIÓN	PROBABILIDAD TI	CONFIANZA	ACCIONES
#10026	Compresor TI	25 jul 2026, 14:32	CSV	No falla	33%	Media	
#10035	Compresor 08	25 jul 2026, 07:32	CSV	No falla	33%	Media	
#10098	Compresor TI	24 jul 2026, 22:32	Manual	No falla	33%	Media	
#10107	Compresor TI	23 jul 2026, 05:32	Manual	No falla	20%	Media	
#10043	Compresor 08	22 jul 2026, 22:32	CSV	No falla	11%	Alta	
#10059	Compresor 01	21 jul 2026, 06:32	CSV	No falla	23%	Media	
#10109	Compresor 08	20 jul 2026, 02:32	Manual	No falla	9%	Alta	
#10071	Compresor TI	19 jul 2026, 22:32	CSV	No falla	23%	Media	
#10073	Compresor 08	18 jul 2026, 14:32	Manual	No falla	20%	Media	
#10178	Compresor 08	15 jul 2026, 07:32	CSV	No falla	4%	Alta	

Mostrando 1-10 de 16 resultados

Figura 6. Historial de predicciones con filtros combinables, que permiten al supervisor auditar resultados por máquina, período, tipo de resultado u origen del dato.

9.7.Propuesta de funcionamiento futuro

En una etapa posterior, el sistema se desarrollaría mediante una arquitectura compuesta por los siguientes elementos:



El frontend enviaría los datos ingresados por el usuario hacia una API REST desarrollada, por ejemplo, con FastAPI. El backend validaría y prepararía la información antes de enviarla al modelo de Machine Learning.

Posteriormente, el sistema devolvería la probabilidad de falla y la explicación de las variables más influyentes. Finalmente, los resultados se almacenarían en una base de datos para alimentar el historial, el panel principal y las métricas de monitoreo.

10. Desarrollo Inicial del Proyecto

El desarrollo se planificó bajo un enfoque iterativo, asegurando que cada etapa tenga responsables y entregas definidas.

Por ello se ha adoptado la metodología CRISP-DM como nuestra hoja de ruta. Este marco de trabajo nos permite avanzar de manera estructurada, pero con la flexibilidad suficiente para ajustar nuestras estrategias si los datos no cuentan una historia distinta a la esperada

10.1. Hoja de ruta

Se ha organizado el desarrollo en fases claras, distribuyendo las responsabilidades para garantizar la trazabilidad y el cumplimiento de los tiempos:

- **Fase 1: Comprensión del Entorno (Semana 1):**
 - Más allá de las variables, nos enfocamos en el “dolor” del usuario. Analizamos qué significa realmente una parada de máquina en la línea de producción y definiremos cuáles son los KPIs de falla que, de ser detectados a tiempo, salvarían costos operativos.
- **Fase 2: El “Artesanado” de los Datos (Semanas 2-3):**
 - Utilizando *pandas* y *matplotlib*, dedicaremos tiempo a limpiar el dataser. Sabemos que los sensores generan ruido y que el desbalanceo de clases es un reto real, por eso, esta etapa es crítica. No solo limpiaremos datos, sino que buscaremos patrones de comportamiento antes de la falla.
- **Fase 3: Entrenamiento y Experimentación (semanas 4-6)**
 - Aquí pondremos a prueba nuestros modelos (Random Forest, SVM y XGBoost). Lo interesante no es solo ver qué modelo acierta, sino entender por qué lo hace. Compararemos métricas clave como el F1-Score, priorizando no dejar fallas sin detectar.
- **Fase 4: Transparencia y Explicabilidad (Semanas 7-8)**
 - Un modelo del tipo “caja negra” no es útil en la industria. Se implementará técnicas de interpretación (como SHAP) para que, cuando el sistema alerte sobre un posible fallo, podremos decirle al operario: *“La temperatura es normal, pero la combinación de velocidad y desgaste de herramienta sugiere un riesgo de sobrecalentamiento”*.

10.2. Dinámica de Trabajo

La gestión es tan importante como la técnica. Para asegurar que este plan se cumpla a cabalidad, se adoptará un método de trabajo bajo los siguientes pilares:

1. **Sprints de Trabajo (Metodología Ágil):** Trabajaremos con ciclos semanales de revisión. Los integrantes del equipo tienen un rol asignado (desde la limpieza de datos hasta documentación), pero nos reuniremos cada semana para sincronizar avances y resolver bloqueos técnicos (especialmente en la configuración del entorno de desarrollo).
2. **Trazabilidad Total:** Todo el código será gestionado en un repositorio compartido garantizando que cada cambio sea auditable. Esto no solo cumple con los estándares de ingeniería, sino que nos permite la colaboración sin “pisarnos” el código.
3. **Gestión de Riesgos Proactiva:** El desbalanceo de los datos es un riesgo latente. Por ello, si en la Fase 2 notamos que el modelo no aprende de las fallas, tenemos planificado pivotear hacia técnicas de oversampling (como SMOTE) antes de avanzar a la fase de modelado.

11. Conclusión

El desarrollo de este proyecto permitió abordar la problemática del mantenimiento predictivo en equipos industriales mediante la aplicación de técnicas de aprendizaje automático y el diseño de una solución informática orientada a apoyar la toma de decisiones. A partir del análisis de la literatura científica se identificó que las estrategias tradicionales de mantenimiento presentan limitaciones importantes, principalmente debido a las detenciones inesperadas, los altos costos de reparación y la dificultad para anticipar fallas. Asimismo, se constató que el aprendizaje automático representa una alternativa capaz de mejorar la planificación del mantenimiento mediante el análisis de datos históricos y variables de operación.

Como respuesta a esta problemática se propuso Perfectime, una plataforma de mantenimiento predictivo diseñada para integrar un modelo de aprendizaje automático con funcionalidades de carga de datos, monitoreo, visualización de resultados e interpretación de las predicciones. Para ello se definieron requisitos funcionales y no funcionales orientados a garantizar la usabilidad, escalabilidad, seguridad y explicabilidad del sistema, considerando además una arquitectura que facilite futuras mejoras y la incorporación de nuevos modelos.

En la etapa experimental se desarrolló un flujo completo de aprendizaje automático utilizando el conjunto de datos AI4I 2020 Predictive Maintenance Dataset. El proceso contempló el análisis exploratorio de los datos, la evaluación de su calidad, el preprocesamiento, el escalado de variables, el balanceo de clases mediante SMOTE y el entrenamiento de los modelos Random Forest, XGBoost y Perceptrón Multicapa (MLP). Posteriormente, los modelos fueron comparados utilizando distintas métricas de evaluación, permitiendo seleccionar XGBoost como la alternativa con mejor desempeño para la detección de fallas industriales.

Además del rendimiento predictivo, se incorporó una etapa de interpretabilidad mediante SHAP, lo que permitió identificar que variables como el desgaste de la herramienta (Tool Wear), el torque y la velocidad de rotación fueron las de mayor influencia en las predicciones. Esto aporta mayor transparencia al modelo y facilita que los resultados puedan ser comprendidos por los futuros usuarios de la plataforma.

Finalmente, este proyecto permitió integrar conocimientos de aprendizaje automático, análisis de datos y desarrollo de software en una propuesta orientada a resolver un problema real de la industria. Si bien la solución presentada corresponde a un prototipo y existen oportunidades de mejora, como incorporar nuevos

conjuntos de datos, optimizar el entrenamiento de los modelos e implementar la plataforma en un entorno productivo, los resultados obtenidos demuestran que es posible desarrollar herramientas que apoyen la planificación del mantenimiento y contribuyan a una gestión más eficiente de los activos industriales. De esta forma, se cumplieron los objetivos planteados para el proyecto y se establecieron bases sólidas para futuras etapas de desarrollo.

12. Bibliografía

- Aminzadeh, H., et al. (2025). A machine learning implementation to predictive maintenance and monitoring of industrial compressors. *Sensors*.
- Benhanifia, M., et al. (2025). Systematic review of predictive maintenance practices in the manufacturing sector. *Results in Engineering*.
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras & TensorFlow* (3.^a ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Guidotti, M., et al. (2025). A systematic literature review of supervised machine learning techniques for predictive maintenance in Industry 4.0. *IEEE Access*.
- Hamasha, M., et al. (2025). A comprehensive framework for IoT-driven predictive maintenance leveraging AI and edge computing for enhanced equipment reliability. *Engineering Science*.
- Meitz, T., et al. (2025). A literature review framework and open research challenges for predictive maintenance. *Computers & Industrial Engineering*.
- Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* 785-794.
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2nd ed.). <https://christophm.github.io/interpretable-ml-book/>
- He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transaction on Knowledge and Data Engineering*, 21(9), 1263-1284. <https://ieeexplore.ieee.org/document/5128907>

- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.